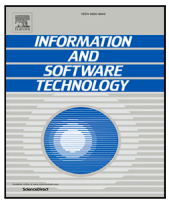
Information and Software Technology 184 (2025) 107753

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof



A systematic literature review on task recommendation systems for crowdsourced software engineering

Shashiwadana Nirmani [a,*], Mojtaba Shahin [b], Hourieh Khalajzadeh [a], Xiao Liu [a]

[a] Deakin University, Burwood, VIC, Australia
[b] RMIT University, Melbourne, VIC, Australia

ARTICLE INFO

Dataset link: CSE-TASK-REC-DATA (Original data)

Keywords:
Crowdsourced software engineering
Task recommendation
Systematic literature review
GitHub
TopCoder

ABSTRACT

Context: Crowdsourced Software Engineering (CSE) offers outsourcing work to software practitioners by leveraging a global online workforce. However, these software practitioners struggle to identify suitable tasks due to the variety of options available. Hence, there have been a growing number of studies on introducing recommendation systems to recommend CSE tasks to software practitioners.
Objective: The goal of this study is to analyze the existing CSE task recommendation systems, investigating their extracted data, recommendation methods, key advantages and limitations, recommended task types, the use of human factors in recommendations, popular platforms, and features used to make recommendations.
Methods: This SLR was conducted according to the Kitchenham and Charters' guidelines. We used manual and automatic search strategies without putting any time limitation for searching the relevant papers.
Results: We selected 65 primary studies for data extraction, analysis, and synthesis based on our predefined inclusion and exclusion criteria. Based on our data analysis results, we classified the extracted information into four categories according to the data acquisition sources: Software Practitioner's Profile, Task or Project, Previous Contributions, and Direct Data Collection. We also organized the proposed recommendation systems into a taxonomy and identified key advantages, such as increased performance, accuracy, and optimized solutions. In addition, we identified the limitations of these systems, such as inadequate or biased recommendations and lack of generalizability. Our results revealed that human factors play a major role in CSE task recommendation. Further, we identified five popular task types recommended, popular platforms, and their features used in task recommendation. We also provided recommendations for future research directions.
Conclusion: This SLR provides insights into current trends, gaps, and future research directions in CSE task recommendation systems such as the need for comprehensive evaluation, standardized evaluation metrics, and benchmarking in future studies, transferring knowledge from other platforms to address cold start problem.

Contents

* Corresponding author.
E-mail addresses: s.dona@research.deakin.edu.au (S. Nirmani), mojtaba.shahin@rmit.edu.au (M. Shahin), hourieh.khalajzadeh@deakin.edu.au (H. Khalajzadeh), xiao.liu@deakin.edu.au (X. Liu).

https://doi.org/10.1016/j.infsof.2025.107753
Received 11 June 2024; Received in revised form 23 February 2025; Accepted 7 April 2025
Available online 22 April 2025

1. Introduction

Crowdsourcing is a distributed problem-solving approach that combines human and machine computation. It involves organizations outsourcing work to an undefined networked labor pool through open calls for participation [1]. Crowdsourced Software Engineering (CSE) is derived from the crowdsourcing concept. CSE utilizes open calls to recruit global online labor in various software engineering tasks, including requirements extraction, design, coding, and testing [1]. It reduces time-to-market by enabling parallelism, lowering costs and defect rates, and enabling flexible development capabilities [2–4]. Mao et al. state that there is a dramatic rise in recent work on the use of crowdsourcing in software engineering based on their conducted pilot study [1].

Popular crowdsourcing platform Topcoder[1] has hosted over 136,202 challenges in software design, development, and data science, with total prizes exceeding US$191,873,091 [5]. uTest[2] has a freelancer pool of more than 1 million testers worldwide who test new apps for device compatibility, conduct functionality testing, and perform usability inspections [6]. As of April 2024, StackOverflow[3] has over 24 million programming-related questions and 36 million answers [7]. Major companies like Netflix, Microsoft, Facebook, and Google frequently offer bug bounties, particularly for security vulnerabilities [8]. With such extensive community involvement, it is crucial to recommend the most suitable tasks to the contributors.

Developers often encounter various technical and social barriers when attempting to onboard crowdsourcing projects [9]. Unsuccessful onboarding not only affects developers but also impacts the projects themselves. It increases the time and effort new developers spend searching for suitable projects and learning [10]. Further, online crowdsourced software development operates as a globally distributed collaborative effort. It involves a diverse range of contributors with varying personalities, educational backgrounds, and levels of expertise. These contributors work from different locations across multiple time zones and undertake various development tasks, such as bug fixing, code testing, or documentation, driven by their individual motivations. Hence, these contributors may struggle to find suitable projects or tasks among the thousands available in these communities [11]. Various recommendation systems have been introduced in the literature to provide personalized task recommendations to software practitioners on crowdsourcing platforms. Providing such recommendations results in increased worker satisfaction, lower churning rate after onboarding, and improved task quality.

We retrieved relevant studies from 6 popular digital databases, including Scopus, IEEE Xplore, and ACM digital library, where we found 6, 20, 15, 5, 3, and 4 primary studies, respectively. After forward and backward snowballing, the total number of primary studies led to 65. We found an existing SLR and a survey that are slightly relevant to the focused domain of this study. However, the SLR by Zhen et al. [12] focuses broadly on software as well as non-software tasks in crowdsourcing, while our SLR provides an in-depth analysis of personalized CSE task recommendations for software practitioners. Similarly, Mao et al. [1]'s survey broadly analyzes crowdsourced software engineering practices without specific research questions or overlapping studies, whereas our SLR focuses specifically on recommending tasks to practitioners.

This SLR aims to consolidate existing information and provide an in-depth understanding of the existing literature on providing personalized task recommendations to software practitioners on crowdsourcing platforms. The findings of this study offer insights for both practitioners and researchers focused on enhancing the CSE task recommendation systems while also laying the groundwork for future research in this field. The key contributions of this SLR are as follows:

- Identifying, analyzing, extracting, and synthesizing 65 highly relevant primary studies.
- Analyzing how software task recommendation in crowdsourcing has been done so far and its current status.
- Identifying key advantages and limitations in the existing studies.
- Providing insights into key future research directions and recommendations for further studies.

The key results of our study revealed that the data used to recommend tasks were sourced from 4 sources: software practitioner's profile, task or project itself, previously contributed tasks or projects available in the crowdsourcing platform and direct data collection via a customized platform. The existing software task recommendation methods can be mainly categorized as content-based, collaborative filtering, and hybrid approaches. Content-based approaches are the most popular category. The existing CSE task recommendation systems offer key advantages such as increased performance, accuracy, optimized solutions, and the ability to provide more comprehensive and diverse recommendations. However, they also have three main limitations: limitations in the dataset, approach, and evaluation. Commonly recommended tasks in CSE platforms, such as GitHub, Topcoder, and Utest, include repository contributions, issue fixes, testing tasks, Good

[1] https://www.topcoder.com
[2] https://www.utest.com
[3] stackoverflow.com

Table 1
PICOC table for research questions.

Population	Software practitioners in crowdsourcing platforms (e.g.; developers, testers)
Intervention	Recommending software tasks in crowdsourcing platforms
Comparison	N/A
Outcomes	Existing software task recommendation methods in crowdsourcing / Human factors that have been used when recommending tasks / Crowdsourcing platforms and features that have been used to recommend software tasks
Context	Crowdsourcing/Task recommendation/ Software Engineering practitioners

First Issues (GFI), and answering questions. Among the various features used in existing systems, issue or task descriptions are the most widely utilized for data extraction. Key future research directions include the need for integrating more human factors into these systems, transferring knowledge from other platforms to overcome data sparsity, and exploring the possibility of integrating these recommendation systems with development workflow tools.

The rest of the paper is structured as follows. Section 2 describes the method used for this systematic literature review. Section 3 presents the demographic information of the presented studies and answers the research questions of this SLR. The threats to validity are discussed in Section 4. Section 5 provides a summary of the existing research. The insights into future research directions are discussed in Section 6. Finally, the conclusions of this study are presented in Section 7.

2. Research methodology

This Systematic Literature Review (SLR) aims to assess the existing research studies on recommending a software task to a software practitioner in a crowdsourcing platform. To conduct this SLR in an unbiased way, we adhered to the well-defined method outlined in Kitchenham and Charters' guidelines [13]. The three primary stages of this research methodology are: (1) defining the review protocol, (2) conducting the review, and (3) reporting the review. The review protocol of this study consists of (i) research questions, (ii) search strategy, (iii) inclusion and exclusion criteria, (iv) study selection, and (v) data extraction and synthesis. The following subsections discuss these steps.

2.1. Research questions

The research questions (RQs) were developed by the PICOC approach introduced by Petticrew and Roberts [14], explained in Kitchenham and Charters' guidelines [13]. This approach was originally developed in the field of Health Sciences but has been adapted for other domains, such as Information Technology. It helps streamline the identification of key elements essential for formulating clear research questions and selecting keywords for search string construction. The PICOC (Population, Intervention, Comparison, Outcomes, Context) table for this SLR is available in Table 1. The intervention is the software technologies that address the issue, the population is the people affected by the intervention, the comparison is the technology used in software engineering that is being compared to the intervention, the outcomes are the factors of importance to practitioners, context refers to the context in which the comparison takes place, the people participating in the research, and the tasks being carried out.

RQ 1: How has the task recommendation in crowdsourced software engineering been done so far and what is its current status?

This research question is studied under four sub-questions mentioned below. It aims to investigate the state-of-the-art methods in task recommendation for CSE. It involves exploring the data sources and types of data extracted for task recommendations, categorizing the existing methods, and examining their advantages and limitations to understand the current landscape. It establishes the current state and methodologies of task recommendation systems, setting the context for subsequent RQs. Further, it lays the foundation for identifying opportunities for improvement.

RQ1.1: What data sources are used and what type of data are extracted to recommend tasks?

RQ1.2: What are the existing software task recommendation methods?

RQ1.3: What are the advantages of the existing task recommendation systems?

RQ1.4: What are the limitations of the existing task recommendation systems?

RQ2: What are the common types of tasks that are recommended to software practitioners?

This RQ focuses on understanding the types of tasks commonly recommended to practitioners in CSE platforms. It highlights the areas where task recommendation systems are currently concentrated and provides insights into the scope and diversity of tasks addressed by existing systems, offering a clearer picture of the industry's focus. This RQ builds on RQ1 by narrowing the focus to the types of tasks addressed by these systems, highlighting industry priorities.

RQ 3: To what extent are human factors used to recommend CSE tasks?

According to Dutra et al. human factors, also known as soft factors, human aspects, social factors, or non-technical factors, refer to the physical or cognitive features, or social behaviors of individuals [15]. Human factors cover a wide range of aspects, including personality, motivation, cognition, communication, decision-making, and cohesion among developers as they carry out their tasks [16]. Avison et al. [17] state that "Failure to include human factors may explain some of the dissatisfaction with conventional information systems development methodologies; they do not address real organizations". Since software development is a human-centered process, human factors significantly impact both the process and its performance [18]. Given the human-centric nature of software development, understanding how human factors are integrated into CSE task recommendation systems helps identify gaps in addressing the social and cognitive needs of developers. Hence, this RQ examines the human factors that are currently being considered in the software crowdsourcing task recommendation systems. It also identifies the human factors suggested to be included as future improvements in the selected primary studies. This RQ explores the integration of human factors, a critical dimension that complements the technical aspects addressed in RQ1 and RQ2.

RQ4:What platforms are currently being used to recommend crowdsourced software engineering tasks?

This RQ identifies the different software crowdsourcing platforms currently being used for task recommendations. It further analyses the scale of the projects used to recommend tasks. The answers to this research question provide insights into popular crowdsourcing platforms and their popularity and suitability for different task recommendation needs.

RQ5:Which features of platforms can be used to recommend crowdsourced software engineering tasks?

Table 2
Alternative search terms used.

Main search term	Alternative search terms
Task Recommendation	task selection/issue recommendation /good first issue/feature recommendation/bug recommendation/bug report assignment/onboarding developer/repository recommendation/project recommendation/question recommendation/crowdtesting
Software Crowdsourcing	software crowdsourcing/crowdsourc* software/github/open source/repository/stackoverflow/stack overflow/topcoder/software engineering/software develop*/open source development/recommendation algorithm/recommendation system

```
TITLE-ABS-KEY ( ( "task recommendation" OR "task selection" OR "issue recommendation" OR "good first issue" OR "feature recommendation" OR "bug recommendation" OR "bug report assignment" OR "onboarding developer" OR "repository recommendation" OR "project recommendation" OR "question recommendation" OR "crowdtesting") AND ( "software crowdsourcing" OR "crowdsourc* software" OR "github" OR "open source" OR "repository" OR "stackoverflow" OR "stack overflow" OR "topcoder" OR "software engineering" OR "software develop*" OR "open source development" OR "recommendation algorithm" OR "recommendation system" ) )
```

Fig. 1. Search query.

This RQ identifies the features of crowdsourcing platforms that can be used to recommend tasks and the data items currently being extracted from them. The answers to this research question provide a comprehensive set of features and data attributes, offering actionable insights for system design and improvement in future studies.

2.2. Search strategy

The search strategy used in this study was designed to retrieve as many relevant studies as possible [13], which is explained in detail in the following sub-sections.

2.2.1. Search method

The automatic and manual searches were carried out to find the relevant primary studies for this SLR. Scopus, IEEE Xplore, ACM Digital Library, Wiley, Science Direct, and Springer Link scientific databases were queried during the automatic search using the search query defined in Section 2.2.2. Then, backward and forward snowballing was done on the finalized relevant primary studies retrieved by the automatic search.

2.2.2. Search terms

The search query was formulated according to the Kitchenham and Charters' guidelines [13]. The PICOC approach (see Table 1) was used to select the key search terms. The search query was finalized after a series of pilot searches and examining the resulting studies. In some studies, it was observed that generic terms like "recommendation algorithm" and "recommendation system" were used instead of any crowdsourcing-related terms within the title, abstract, and keywords, e.g., "good first issue recommendation system". Hence, the aforementioned two terms were also added with the alternative search terms for "software crowdsourcing". The final search query was formulated by connecting the alternative search terms with "OR" and "AND", which are mentioned below. The alternative search terms used in the query are mentioned in Table 2. The wild cards were used except in the Science Direct database to cover a large number of alternative search terms. In Science Direct's advanced search, only 8 operators (e.g., AND/OR) are allowed per query. Hence, the combinations of the search terms were used to maintain consistency among the search queries used in different databases. Further, the search was not restricted by any particular period. Finally, we randomly selected 8-10 papers from each digital database to confirm the credibility of our search string and validate that the included studies were the most pertinent to our review (see Fig. 1).

2.2.3. Data sources

The search engines of scientific databases Scopus, IEEE Xplore, ACM Digital Library, Wiley, Science Direct, and Springer Link (Table 3) were used with the filters for automatic search. These are the main sources for potentially relevant research on software and software engineering [19]. Since the Springer Link database allows scanning only the title or the entire document, it returns a significant number of results for the query. Hence, only the top 300 results were analyzed. Since Google Scholar may return imprecise search results that may produce a large number of irrelevant results, and significantly overlap with ACM and IEEE on software engineering literature, it was not included in the data sources [19].

2.3. Inclusion and exclusion criteria

The inclusion and exclusion criteria applied to every study retrieved from digital libraries are shown in Table 4 and Table 5, respectively. These criteria were refined during the search and paper filtering procedures to obtain an unbiased selection of studies. The finalized set of criteria was applied to each full-text article to choose the most relevant studies that align with the research questions and the aim of this SLR. All review papers and grey literature were excluded according to the SLR primary studies screening conventional procedure.

2.4. Study selection

The study selection was done in 4 phases. The breakdown of results for each phase according to the search method and library is available in Table 6. Fig. 2 shows the number of studies retrieved at each phase of the study selection process.

Phase 1: The potentially relevant studies were retrieved from the chosen digital libraries using the designed search query. The total number of potential results for the query was 1368 (i.e., only the top 300 results were considered for the Springer Link database). Then, the inclusion and exclusion criteria were applied to the relevant studies set that had been retrieved.

Phase 2: During the second phase, the potential studies set was screened by reading the title, abstract, and keywords. The papers which we had doubts regarding the relevancy were transferred to the next round of screening. The final count of potential studies at the end of the first screening round was 142.

Phase 3: During the third phase, the filtering was done by reading the full-text papers of the potentially relevant studies found during the second phase. If the proposed solution of the study is not to recommend a software engineering-related task to a software practitioner or instead

Table 3
Queried scientific databases.

Database	Search Method	Filters Used
Scopus	Advanced search	TITLE-ABS-KEY
IEEE Xplore	Command search	All Metadata
ACM Digital Library	Advanced Search	Title, Abstract, Author Keyword filters added separately
Wiley	Advanced search	Title, Keywords, Abstract filters added separately
Science Direct	Advanced Search	Title, abstract or author-specified keywords. Does not support wildcards, hence removed the wildcards. Has 8 operators (AND/OR) per query limitation. Hence used combinations of the search terms.
Springer Link	Advanced Search	Disciplines: Computer Science

Table 4
Inclusion criteria.

ID	Criteria
I01	Peer-reviewed published studies that are available in full text
I02	Studies that are focused on recommending software engineering-related tasks to a software practitioner in a crowdsourcing platform
I03	Study is published in one of the digital libraries we queried for this study

Table 5
Exclusion criteria.

ID	Criteria
E01	Unpublished and incomplete work, posters, and grey literature were excluded
E02	Studies that are focused on recommending a software practitioner to do a software engineering-related task published in a crowdsourcing platform
E03	Papers that are not written in English language
E04	Short papers which have less than 4 pages
E03	Papers that do not propose a recommendation system based solution

it is to recommend a software practitioner to do a task in a CSE platform, such studies were excluded in this stage after a careful evaluation of the proposed solution. The data extraction of the primary studies was also done during this stage. After this filtering round, 51 papers were left as the primary studies.

Phase 4: The snowballing was done during the last phase for the finalized primary studies. Both backward and forward snowballing processes were used, and 9 and 12 potential studies were found, respectively. Then, the first filtration for those studies was done based on the title, abstract, and keywords. Then, 6 papers were left for backward snowballing, and 8 were left for forward snowballing at the end of filtration round 1. Then during the next filtration round, we read the full-text papers of those studies. After the second filtration, the total number of relevant primary studies finalized by backward snowballing was 5, and forward snowballing was 7. The finalized set of primary studies are available in Appendix A.

2.5. Quality assessment

We implemented a five-point scoring system to evaluate the quality of the selected primary studies based on five predefined questions. The scores range from very poor (1) and inadequate (2) to moderate (3), good (4), and excellent (5). This approach to quality assessment is widely used in SLRs [20,21]. Our quality assessment (QA) criteria were derived from Kitchenham's guidelines [13] and are outlined as follows:

- QA1:Are the aims clearly stated?
- QA2: Are the measures used clearly defined?
- QA3: Is the solution clearly defined?
- QA4: Is there a clear outcome and results analysis reported?
- QA5: Are study limitations and possible future work adequately described?

The results of this assessment are presented in Appendix B. Our assessment revealed that among the 65 included studies, 14 were of good quality, 39 were of average quality, and 12 were of poor quality. No studies were excluded based on the quality assessment score as recommending CSE tasks to practitioners is an emerging research area, and our selection was already constrained. We did not consider the citation count of each paper as a quality criterion to minimize publication bias and to ensure fairness toward recently published studies.

2.6. Data extraction and synthesis

The data items extracted from the selected primary studies to answer the research questions of the SLR are presented in Table 7. The data items were retrieved and categorized based on each research question. The extracted data were collected in a Microsoft Excel Spreadsheet file. Data synthesis of the extracted data was done by thematic analysis [22,23]. This approach is often employed in systematic literature reviews (SLRs). It can be defined as an analysis of an extensive amount of structured, extracted data from individual studies to integrate and synthesize their main findings. The analysis was conducted by the first researcher, while the second researcher reviewed all extracted themes

Table 6
Breakdown of results for the paper selection phases.

	Database	Initial results count	Filter by Title, Abstract, and Keywords	Filter by Full text reading
Automatic Search	Scopus	836	85	6
	IEEE Xplore	98	26	20
	ACM Digital Library	32	19	15
	Science Direct	16	5	5
	Wiley	86	3	3
	Springer	1213 (top 300 analyzed)	4	4
Total - Automatic Search		**1368**	**142**	**53**
Manual Search	Backward Snowballing	9	6	5
	Forward Snowballing	12	8	7
Total - Manual Search		**21**	**14**	**12**
Final Paper Count				**65**

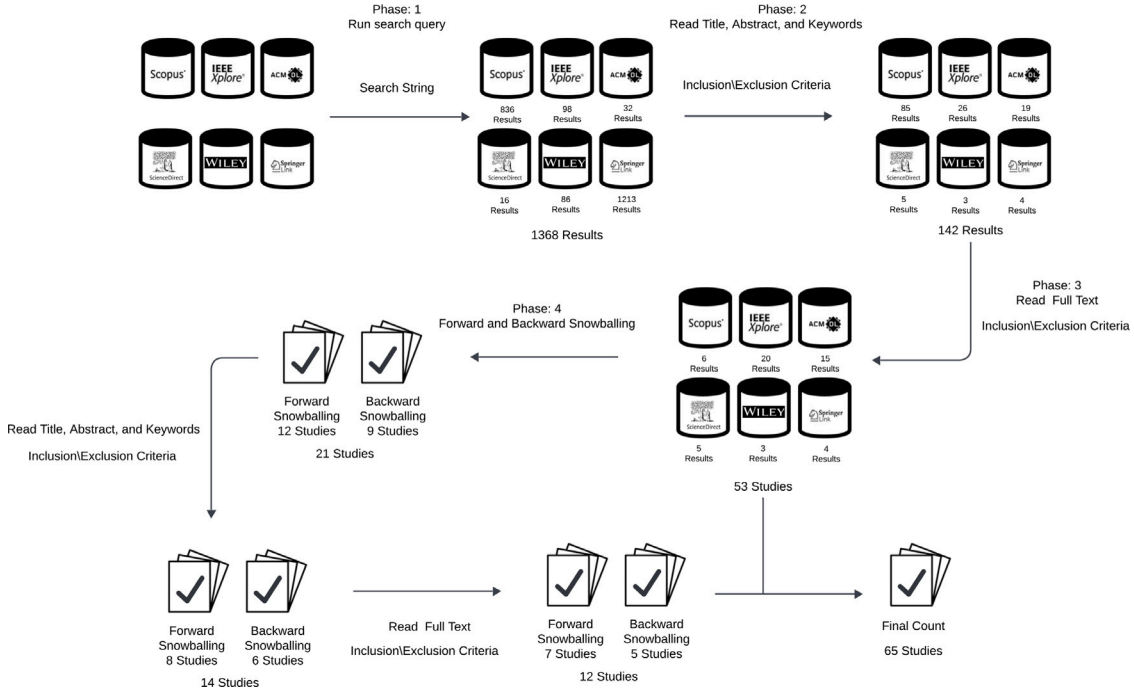


Fig. 2. Phases of study selection process.

to validate them and identify any additional potential themes. We performed the thematic analysis using the five steps outlined by [23].

1. Familiarizing with data: The extracted data items, such as task types (D08), advantages (D06), limitations (D07), data extracted from the user (D02), and data extracted from the task (D03), were carefully read and examined to develop an initial understanding.
2. Generating initial codes: Lists of advantages, limitations, data extraction sources, etc., were compiled for each solution. In some cases, the original papers were revisited to ensure accuracy (Fig. 3.A).
3. Searching for themes: The related initial codes from the previous step were grouped to form potential themes for each data item (Fig. 3.B).
4. Reviewing themes: Identified themes were compared and analyzed, assessing whether some needed to be merged or removed.
5. Defining and naming themes: Finally, clear and meaningful names were assigned to each identified data extraction source, advantage, limitation, etc., ensuring they accurately represented the findings (Fig. 3.C).

Fig. 3 shows the application of thematic analysis to analyze various advantages and identify the "Making more comprehensive diverse recommendations" theme.

3. Results

The analysis and synthesis of the data extracted from our finalized 65 primary studies are reported in the following subsections. The results discussed in this section are derived by synthesizing the data with minimal interpretations directly extracted from the selected primary study set.

3.1. Demographic data

Of the 65 primary studies selected for this SLR, 37 (56.9%) were conference papers, 26 (40%) were journal papers, and 2 (3.1%) were workshop papers. Fig. 4 shows the yearly distribution and publication type distribution of the selected papers. The selected primary studies were published between 2011 and 2024 era even though we did not limit our search to a specific period. None of the included studies were published in 2012. There is an overall increase in the number of publications from 2011 to 2018. Then, in 2019 and 2020, there was a

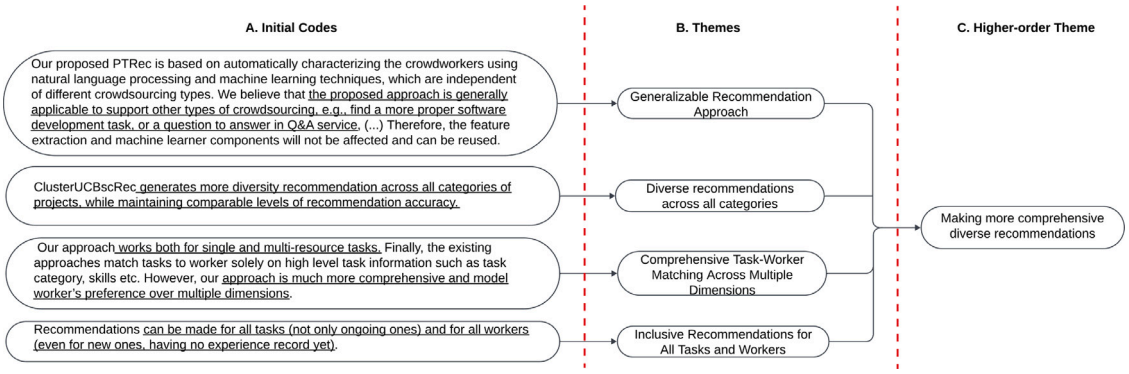


Fig. 3. The steps of performing thematic analysis.

Table 7
Data items extracted.

	Data Item	Description	RQ
D01	Title	N/A	RQ1
D02	Type of data extracted from user	What kind of data extracted from the user (eg: preferences)	RQ1
D03	Type of data extracted from task	What kind of data extracted from the task (eg: skills required)	RQ1
D04	Method steps	Implementation steps of the proposed system	RQ1
D05	Method type	The algorithmic category of the proposed system (eg: Matrix factorization)	RQ1
D06	Advantages	Advantages of the proposed system	RQ1
D07	Limitations and future work	Limitations and the future work of the study	RQ1
D08	Task type	Type of the task recommended	RQ2
D09	Human factors used	Human factors included in the study	RQ3
D10	Human factors suggested	Human factors suggested to be included as a future work of the study	RQ3
D11	Features	Features of the crowdsourcing platform used to extract data (eg: ReadMe file)	RQ4
D12	Extracted data	Data extracted from the features (eg: user activities like star, fork)	RQ4
D13	Platform	Name of the crowdsourcing platform used in the study (eg: Github)	RQ5
D14	Project name(s)	Name(s) of the projects that were used in the study	RQ5
D15	Comments	Additional information	N/A
D16	Author(s)	Author(s) names	Demographic data
D17	Year	Publication year	Demographic data
D18	Venue	Publication venue	Demographic data

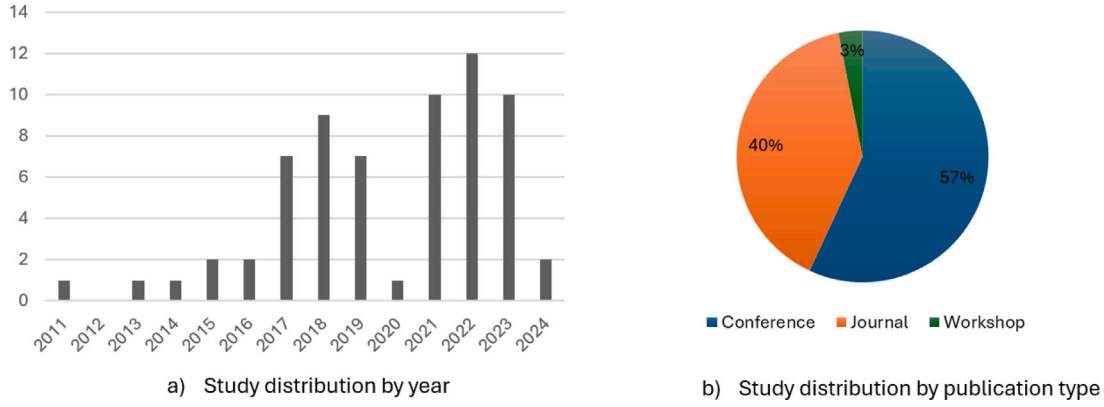


Fig. 4. Publication trends.

drop in the number of publications. The COVID-19 outbreak might have caused the sudden publication drop seen in 2020. However, it is not certain. Overall, 52%of the selected primary studies set were published from 2021 to 2024.

3.2. RQ1: How has the task recommendation in crowdsourced software engineering been done so far and what is its current status?

This RQ aims to analyze how software task recommendations in crowdsourcing platforms have been made to date. To answer this RQ, we analyzed the data extracted to recommend tasks, proposed methods, the advantages, and the limitations of the proposed methods. The results of the analysis are discussed in the following sub-sections.

3.2.1. RQ1.1: What data sources are used and what type of data are extracted to recommend tasks?

After analyzing the primary studies, we identified that the data used to recommend tasks in the selected primary studies were sourced from 4 sources, described below. Table 8 shows the extracted data according to the data acquisition source.

1. **Software Practitioner's Profile**: One of the main sources of data is the software practitioner's profile. Such profile is mainly used to extract the data listed below:

    - *Skills* - Practitioner's skills, such as the programming languages worked with, tools, and platforms familiar with,

were extracted from their profiles. For example, Study S14 extracted software practitioner's skills in programming languages, frameworks, platforms, etc from the practitioner's profile.
- *Ratings* - Ratings received for previously completed tasks reflect the expertise level of the practitioner. For example, Study S29 extracted ratings received for previously completed tasks as ratings capture how well a worker has worked on other criteria in the past.
- *Workload* - It is important to evenly distribute the newly reported bugs among potential developers who are not overloaded by work. Study S48 extracted the developer's work history from the developer's profile to decide on the current workload.

2. **Task or Project**: The task or project itself provides insights about the job required to be completed. Data listed below are extracted from the task or project itself.

    - *Skills, expertise* - The skills and expertise required to complete the task are popular data types available for the tasks or projects. Study S5 and S27 extracted the programming language from the task to determine the required skills to complete the task.
    - *Task's competitive advantage, Technologies hotness* - Study S2 captures the task's competitive advantage over the other open tasks. Technologies hotness refers to the relative level of demand and supply of a particular technology. Study S39 defines the hotness of technology on a five-point scale from 1 (most popular) to 5 (least popular) for the technologies mentioned in the task requirements.
    - *Progress, Status, Severity* - These data items provide quick insights into the task. The status of a task is often described using different tags as 'fixed', 'invalid', 'duplicate', 'wontfix', and 'worksforme' (e.g., S42 extracted 'fixed' or 'verified' status from the task). The severity describes the importance using 'critical', 'major', etc tags. (e.g., S48 extracted the severity from the bug reports)
    - *Resources* - The resources allocated for the task like the time and budget are extracted from the task. For example, Study S15 captured the budget and estimated completion time. Study S28 extracted the task hours and rates from the task poster.
    - *Domain* - The domain refers to the specific area the task or project focuses on. Study S15 extracted project category and type from the posted task.
    - *Code involved* - The code involved in the required fix is another data extracted from the task. Study S7 used the source code when building the feature vector of keywords used to recommend the task.
    - *Popularity* - The task or project popularity is another popular data extracted. Study S17 extracted the number of stars received and the number of forks which provides insights about the popularity of the repository among the community.
    - *Feedback* - online feedback of the project sponsors and developers on the task was extracted in Study S15.
    - *Task description sentiments* - Study S9 extracted the sentiments associated with task description such as negative and positive emotions as they impact newcomers selecting which issues to work on.
    - *Contributor's social characteristics* - Social characteristics of the contributors, such as influence gained from developer's friends to bid for similar tasks, and the frequency of interaction with other users were gathered from the task or the project itself. For example, In Study S8, inactive developers received recommendations by pooling together task recommendations from their friends who bid on tasks.

3. **Previous Contributions**: Some studies extracted data from the software practitioner's previously contributed tasks or projects as they provide insights into the practitioner's preferences, skills, expertise, etc.

    - *Skills, Expertise* - Skills and expertise level required to complete the task were extracted from previous contributions to gather data about the practitioner's past skills and expertise. For example, in Study S2, expertise is considered as a crowd tester's capability of detecting bugs. Hence, it is extracted based on the number of bug reports submitted by the worker for each domain.
    - *Activeness* - Study S2 extracted the number of bugs or reports submitted within a specified period to measure the activeness of testers.
    - *Historical behavior sequences* - Historical behavior sequences refer to creating a pull request, starring, forking, watching a repository, commenting, bidding for a task, etc. These are considered reflections of the developer's programming preferences or interest in the project. For example, Study S17 extracted 'Create', 'Fork', 'Star', and 'Pull' Request activity data from the developer's previously contributed projects.
    - *Sentiments exhibited in communication* - Study S59 proposes that newcomers exhibiting a more positive communication style are more likely to receive assistance from experienced developers when dealing with complex issues. Further, Newcomers with less coding experience and who express negative sentiments than their peers tend to prefer adaptive issues over corrective or perfective ones. Those who contributed to the first issues in web library and framework projects generally display more positive sentiment characteristics. Hence, they extracted mean and median textual sentiment polarity values of previously contributed Pull requests and reported issues.
    - *Task intensity or complexity, Severity* - Task intensity or complexity (strong, weak, etc.), severity (critical, major, normal, etc.) were also extracted from previous contributions. Study S10, extracted intensity (strong, medium, weak, etc.) and severity data (critical, major, etc.).
    - *Contributed source files or components* - Source files refer to the individual code files that are related to a specific feature, while components are distinct modules that group related issues and functionalities within the project. Study S3, extracted the previously contributed components of the software projects by the developer.
    - *User role evolution* - According to Study S49, each user can take on the roles of both asker and answerer at the same time in question answering crowdsourcing platforms. However, user roles vary over time. Hence, they extracted the software practitioner's simultaneous role evolution as a question-asker and answerer in the crowdsourcing platform.

4. **Direct Data Collection**: In some cases, a customized platform is introduced as a website to directly gather the required specific data, listed below, from the software practitioners. The main reasons for introducing these platforms to gather data include: existing crowdsourcing platforms not storing some required data (e.g., personality types) and none of the platforms being open to providing information on their developers or clients [24,25].

    - *Demographic data* - Demographic data such as education level, location, etc were gathered by a new website in Study S4.
    - *Personality type* - Studies S4 and S57 gathered personality types of the developers.

Table 8
Extracted data according to the data acquisition source.

Data acquisition source	Gathered data	Studies
Software Practitioner's Profile	Skills	S14, S15, S16, S26, S27, S29, S46, S48, S53, S59
	Ratings	S29
	Workload	S48
Task or Project	Skills, Expertise	S1, S5, S12, S15, S27, S34, S52, S55, S62
	Task's competitive advantage, Technologies hotness	S2, S39
	Progress, Status, Severity	S2, S3, S10, S42, S43, S48
	Resources (time, money)	S4, S15, S28
	Domain	S4, S10, S14, S15, S23, S26, S30, S31, S35, S36, S40, S46, S54, S57, S58, S59
	Code involved	S7, S41, S50
	Popularity	S14, S17, S18, S26, S36, S51, S56
	Feedback	S15
	Task description sentiments	S9
	Contributor's social characteristics	S8, S12, S34, S36, S38, S53, S65
Previous Contributions	Skills, Expertise	S2, S5, S10, S25, S27, S28, S29, S32, S37, S39, S40, S41, S42, S43, S45, S46, S52, S55, S64, S65
	Activeness	S2
	Historical behavior sequences	S5, S7, S8, S10, S17, S18, S19, S20, S21, S22, S23, S24, S32, S36, S47, S50, S52, S55, S60, S65
	Sentiments exhibited in communication	S59
	Task intensity or complexity, Severity	S10, S33
	Contributed source files or components	S3, S13, S48
	User role evolution	S49
Direct Data Collection	Demographic data	S4
	Personality types	S4, S57
	Project and skill preferences	S1

- *project and skill preferences*- Study S1 introduced a website that enables users to select a project and input their skill set to receive personalized recommendations.

RQ1.1 Summary: The data used to recommend tasks in the selected primary studies were sourced from 4 sources: software practitioner's profile, task or project itself, previously contributed tasks or projects available in the crowdsourcing platform, and direct data collection via a customized platform. The most popular data type extracted from these sources is the software practitioner's historical behavior sequences, such as starring, forking, and watching a repository.

3.2.2. RQ 1.2: What are the existing software task recommendation methods?

To categorize the existing task recommendation methods of the primary studies, we used the taxonomy introduced in [26]. Hence, the recommendation systems in the primary studies were broadly categorized into *content-based*, *collaborative filtering*, and *hybrid recommendation systems* categories.

Content-based. In *content-based* recommendation systems, all data items are gathered and categorized into distinct item profiles according to their descriptions or features [26]. For example, Study S3 created a user profile for each developer based on the previously fixed issues. Whenever a new issue arrived, they compared the 30 most frequent words or tokens of the developer's profile with TF-IDF (Term Frequency - Inverse Document Frequency) values of all tokens of the new issue's preprocessed title and description to determine if the new issue falls in the developer's preferred task category. Content-based approaches are the most popular category of the selected primary study set (65% of the primary study approaches). Further, to sub-categorize the content-based systems, we used the taxonomy introduced in [27]. Hence, the content-based category was further subcategorized into the following sub-categories:

- *Machine Learning*: Deciding whether to recommend an item to a user can be viewed as the problem of modeling a task and predicting if the user will like it. To achieve this, different standard machine learning techniques can be used to predict if a user will be interested in an item or not [27]. For example, Study S2 extracted 60 features automatically and employed a random forest learner to generate dynamic and personalized task recommendations aligned with a worker's skills and preferences. These features encompass the task's progress status, crowdworkers' availability, the matching degree between the task and crowdworkers, as well as the task's competitive advantages among other open tasks. From the content-based approaches, 56.4% are machine learning based approaches. We further categorized the selected primary studies into the below machine learning categories mentioned in the taxonomy introduced in [27].
  - *Classification* - Classification algorithms learn decision boundaries and separate instances in a multi-dimensional space [27]. For example, Study S1 used the Random Forest Classifier to determine relevant domain labels for the issues based on the inputted issue text.
  - *Artifical Neural Networks (ANN)* - Artificial Neural Networks (ANNs) are models inspired by the behavior of biological neurons. They attempt to mimic the brain's way of processing information and learning to some extent [27]. Study S17 used a multi-layer graph convolutional network to recommend repositories.

– *Reinforcement Learning* - Reinforcement learning is the problem faced by an agent that learns behavior through trial-and-error interactions with a dynamic environment [28]. The machine learning category was further extended by us by adding the reinforcement learning category to the existing taxonomy. Study S15 introduced a clustering-based reinforcement learning solution to recommend tasks to newcomers.

- *Vector-based Representation*: This is based on the idea that a simple way to describe an item is by listing its specific attributes or features. When user preferences are expressed based on these features, the recommendation task involves matching them accordingly [27]. For example, Study S11 recommended similar bug reports by calculating three similarity scores based on the bug titles and descriptions, product, and component information for the query bug and each of the pending bugs. In the final step, they merged the three similarity scores to generate a final score for each pending bug and recommended the bugs with the highest final scores as the ones most similar to the query bug. From the content-based approaches, 43.6% are vector-based representations.

Collaborative Filtering. These approaches rely on determining the similarity between users [26]. This method begins by identifying a group of users, denoted as X, whose preferences, interests, and dislikes closely resemble those of user A. 20% of primary studies are collaborative filtering approaches. Further, these methods can be categorized as follows:

- *Model-based*: Model-based systems use gathered information to build a model that generates recommendations [27]. They employ various data mining and machine learning algorithms to create a model that predicts a user's rating for an unrated item. Instead of relying on the entire dataset to generate recommendations, these systems extract features from the dataset to build the model [26]. For example, Study S8 built a developer social network and then used singular value decomposition (SVD) to recommend tasks to active developers. Of the collaborative filtering-based primary study approaches, 75% are model-based.
- *Memory-based*: Memory-based approaches recommend new items based on the preferences of the neighborhood. They mostly use similarity metrics to measure the distance between two users or two items [27]. These methods directly utilize a utility matrix for making predictions. The first step involves constructing a model, which is essentially a function that uses the utility matrix as input. Recommendations are then generated through a function that takes both the model and the user's profile as input [26]. For example, Study S31 computed a similarity matrix for test tasks using past test tasks completed by all crowdworkers. Then, based on this similarity matrix and a crowdworker's history of test tasks, their method determines the crowdworker's preference for other test tasks for recommendations. 25% of the collaborative filtering-based primary study approaches are memory-based.

Hybrid Approach. *Hybrid methods* might integrate outcomes obtained from distinct methods, it could apply content-based filtering alongside collaborative methods, or use collaborative filtering within a content-based approach [26]. As an example, Study S27 is a combination of developer profiles, topic distributions of the crowdsourcing tasks (content-based filtering), and a social network to measure a developer's influences (collaborative filtering). 15% of the primary studies use hybrid approaches. Fig. 5 shows the extended taxonomy for the recommendation systems introduced in the primary studies.

RQ1.2 Summary: The existing software task recommendation methods can be categorized as content-based, collaborative filtering, and hybrid approaches. Content-based approaches are the most popular category, accounting for 65% of the approaches. Collaborative filtering-based approaches come in second by 20%, followed by hybrid approaches by 15%.

3.2.3. RQ1.3: What are the advantages of the existing task recommendation systems?

After analyzing the advantages of each primary study (i.e., data item D06), we identified three main categories of advantages. These categories are discussed in detail in the following subsections.

Increasing performance, accuracy, and optimizing solutions: The performance, accuracy, and system optimizations in the selected primary studies were mainly achieved by using two or more characteristics mentioned in Section 3.2.1. For example, studies S8, S12, S34, S36, S38, S53, and S65 used software practitioner's social connections with fellow software practitioners along with other characteristics to improve the accuracy of the systems. According to Study S12, the use of social metrics in their recommendation system improved the precision of the used models up to 15.82%. Further, some studies (e.g., S5, S7, S27) used hybrid recommendation systems (mentioned in Section 3.2.2) to mitigate the limitations that occurred when using one recommendation approach.

Making more comprehensive diverse recommendations: Making more comprehensive diverse recommendations involves offering a wide range of recommendations that cater to different platforms, activities, or user preferences. For example, Study S2 is based on the automatic characterization of crowd workers through the use of machine learning and natural language processing techniques. Hence, those are not dependent on the specific types of crowdsourcing. Even though the system is focused on crowd testing, the proposed method can be used to support other crowdsourcing activities such as finding a better software development task or a question to answer in a Q&A platform.

The recommendation system proposed in S15 can make diverse recommendations across different project categories. The proposed system achieves it by analyzing feedback from users and behaviors which results in learning the preferred projects and developers' skill levels. The recommendation system proposed in study S28 can make more comprehensive recommendations as it models workers' preferences over multiple dimensions. In Study S39, the recommendations can be made to all tasks, not just those that are currently in progress, and to every employee, including those who are new and lack prior experience. The system analyses worker's preference patterns, technologies' hotness, and the projection of winning chances to make recommendations.

Providing recommendations to newcomers ("cold start" users): The "cold start" problem refers to the challenge of providing recommendations to new users who do not have enough historical data or activity to generate accurate recommendations. Some of the existing literature can provide recommendations to cold-start users. Study S8 addresses the "cold start" problem by integrating the recommended tasks of their taskbidding friends. Study S9, automatically identifies issues that newcomers can fix by analyzing the history of issues that have been resolved by using the title and description of issues. The method proposed in S14 uses a topic sampling-based genetic algorithm to recommend tasks to newcomers and transforms the cold-start developer recommendation problem into a multi-optimization problem. Study S15 addresses the "cold start" problem by learning about project preferences and development skill levels of the software practitioner through analyzing and mining user feedback and behaviors.

Study S28 solves the "cold start" problem by modeling worker's preferences over multiple dimensions. Study S44 recommends tasks to

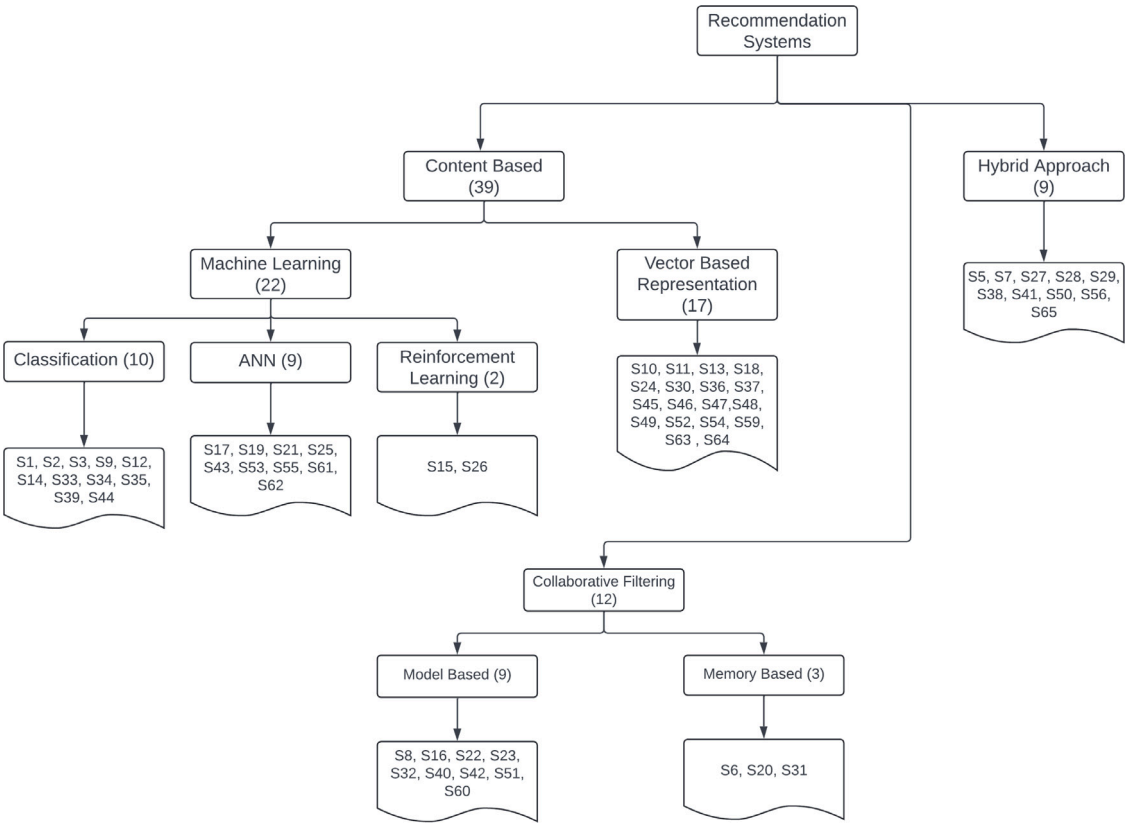


Fig. 5. Taxonomy for proposed methodologies.

newcomers by integrating repository history data (such as the number of prior commits made by the issue reporter at the time of the issue's creation) and global GitHub profiles for all issue reporters and repository owners (e.g., stars received, commits). Study S59 recommends tasks to cold-start users by analyzing possible features and the characteristics of the newcomers' chosen first issues. A new cross-platform recommendation approach for software crowdsourcing was introduced in S61 by transferring data and knowledge from other popular software crowdsourcing platforms. Study S61 adopts a transfer learning approach to solve the platform "cold start" problem.

RQ1.3 Summary: There are several advantages in the existing CSE task recommendation systems, such as increased performance, accuracy, and optimized solutions, making more comprehensive and diverse recommendations, and providing recommendations to newcomers ("cold start" users).

3.2.4. RQ1.4: What are the limitations of the existing task recommendation systems?

We analyzed the limitations pointed out in the primary studies and categorized those limitations into three main categories: dataset, approach, and evaluation. There were no any mentioned limitations in Studies S5, S6, S7, S14, S19, S22, S25, S29, S41, S46, S57, S58, S60 and S63. The limitations are discussed in detail in the following sub-sections.

(i) **Limitations in the datasets (41 papers):** In several primary studies, the dataset used in the primary studies has limitations. Due to the nature of the available information on crowdsourcing platforms, some factors that could be considered for recommendations were ignored. This limitation can lead to inadequate or biased recommendations, as crucial factors influencing recommendation quality, such as the difficulty of tasks or the activity levels of developers, may not be captured. For example, no information indicates task difficulty/ease level in the dataset used in Study S2. Study S53 could not extract several features that affect developers' project selection, like the mailing list and download count, due to the nature of the data they obtained from GitHub. Making recommendations for inactive developers is challenging due to the relative scarcity of available information about them. As a result, the proposed metrics in Study S20 need to be evaluated using other datasets. Consequently, the recommendations may lack accuracy and relevance, potentially resulting in the inability to recommend the most suitable task for the software practitioner. The data set used in Study S39 was extracted from Topcoder. Hence, the conclusions they made might not be generalizable to other platforms.

(ii) **Limitations in the approaches (7 papers):** One of the key limitations identified in the proposed approaches is the lack of generalizability of the solution. The recommendation model in Study S3 primarily follows a content-based approach, limiting its ability to suggest issues to developers based solely on past contributions. It lacks the capability to provide more insightful recommendations that consider serendipity aspects across various components of the same product. Hence, the recommendations could be narrow. The NLP suggestions used in Study S12 might not work as expected in other languages like C++. Therefore, the approach cannot be utilized in projects using different programming languages. Whereas in Study S9, the projects within the scope of the study exhibit diverse practices, backgrounds, resources, and goals. These factors are not adequately considered during the model-building process. Hence, the recommendation system proposed in the study may not be applicable or valid across a broader range of projects with different contexts. Another key limitation is that some systems cannot recommend tasks to newcomers ("cold start" problem).

In Studies S2, S14, S15, S17 and S65, the proposed recommendation systems cannot recommend tasks to newcomers. As mentioned earlier in Section 2.1, human factors play a major role in the software engineering context. However, only 24 primary studies have considered human factors in their recommendation systems. These human factors are analyzed in Section 3.4. 39 primary studies ignored human factors in their recommendation systems. Some primary studies that ignored human factors are, S3, S6, S7, S10, S11, S14, S17, S20, 21, S22, etc.

(iii) **Limitations in the evaluation process (4 papers):** Study S12 restricted predictions for issues with linked Pull Requests (PRs). Hence, it may have inadvertently introduced bias and overlooked significant issues that lacked associated PRs. This approach could have led to incomplete or skewed ground truth data, thereby affecting the accuracy and reliability of their prediction model. Study S13 only used the Spring Framework as the experimental project, and selecting only 15 developers as subjects might limit the generalizability of the proposed approach. This approach risked neglecting the diverse practices and contexts prevalent in other software projects, potentially undermining the relevance and applicability of the recommendations in real-world scenarios. Study S26's reliance on users with specific contribution thresholds and reputation scores (users who have made at least more than or equal to 5 contributions and a reputation score above 10) could have overlooked valuable insights from less active or newer contributors, leading to a biased understanding of users. Hence, the work is limited to the users they studied.

RQ1.4 Summary: There are three main limitation areas in the existing studies: limitations in the dataset, limitations in the approach, and limitations in the evaluation. Limitations in the dataset include overlooking crucial factors like task difficulty, potentially leading to biased recommendations. Proposed approaches often lack generalizability, such as content-based models providing narrow suggestions or NLP methods being programming language-specific. Limitations in the evaluation may introduce bias or limit generalizability, like focusing on specific project contexts or user thresholds, undermining the relevance of findings. These limitations underscore the challenges in developing robust and comprehensive recommendation systems for software practitioners.

3.3. RQ2: What are the common types of tasks that are recommended to software practitioners?

Table 9 shows the types of tasks recommended to software practitioners in crowdsourcing platforms.

- *Repository to contribute*: It refers to recommending a software repository where practitioners can make different contributions such as fixing bugs, adding features, or making improvements. 42.8% of the selected primary studies are to recommend a repository that the software practitioner can contribute to.
- *Fixing issues*: Recommending an issue that practitioners can fix is another type of task recommended to the practitioners. 21.4% of the tasks in primary studies are to recommend issues to fix.
- *Testing tasks*: It refers to recommending a testing task to a software tester. 6.6% of the tasks in primary studies are to recommend software testing tasks.
- *Good First Issues (GFIs)*: Another type of task is called GFIs, i.e., beginner-friendly issues within software projects that newcomers can tackle as their first contributions. 6.6% of the tasks in primary studies are to recommend GFIs.
- *Question to answer*: It refers to recommending a question from a Q&A platform to answer. 6.6% of the tasks in primary studies are to recommend questions.

RQ2 Summary: Recommending a repository that the software practitioner can contribute to, tasks to fix issues, testing tasks, Good First Issues (GFI), and questions to answer are the most common task types recommended in the existing literature. 42.8% of the selected primary studies focus on recommending a repository to contribute.

3.4. RQ3: To what extent are human factors used to recommend CSE tasks?

Human factors encompass personality traits, social and behavioral preferences, and sentiments, which are intrinsic to individuals and require different methods of measurement and analysis. To answer this research question, we analyzed the human factors (i.e., D09 and D10 in 8) currently used in the existing recommendation systems and the human factors suggested to be included as future improvements to the recommendation systems. Table 10 shows these human factors.

Topic and task type preference is used in 9 studies. It is the most used human factor in the existing studies. In study S2, task preference measures the extent to which a prospective task might capture the interest of a potential crowdworker. The worker's willingness and potential to detect bugs increase with their preference of taking up a task. Study S35 will incorporate the preferences of the newcomers as future improvements to their recommendation system.

Skill preference is the second most used human factor used in 6 studies. For example, Study S1 allows a user to select a project to work on, enter their skill set (in terms of predefined categories), and get a list of open issues that would need those skills.

Social and behavioral preferences are another preference type considered in the existing studies. For example, Study S8 introduced a social influence based method to recommend tasks. Study S5 suggests using the social connections of software practitioners as future improvements to their recommendation system.

Studies S4, S57, and S58 investigated the impact of *personality* on task selection in crowdsourcing software development. In study S4, an empirical experiment was carried out to assess the impact of personality on task selection based on the three key components of a task: type, money, and time [24]. Study S57 was carried out to develop a task assignment algorithm for crowdsourcing software development that takes task types and personality types (e.g., Introversion and Extroversion, etc.) into account. The proposed method functions in the same way as the open call format, but it incorporates personality-based categorization [25]. To identify the personality types of software practitioners, the Myers-Briggs Type Indicator (MBTI) was used [29]. MBTI is a popular tool used to analyze personality types in the workplace, which measures a person across four dimensions: energizing, attending, deciding, and living [30]. Study S58 was carried out to find the relationship between testing technique, tester, and personality type and identify which personality types of testers are more competent at a particular testing approach [31]. They also used the MBTI to identify the personality types [30].

Study S9 incorporated the assumption that *sentiments* can influence newcomers in selecting issues to work on. Hence, they calculated the sentiments associated with the descriptions of the unprocessed issue. It returns two scores because people might express both positive and negative emotions in the same line, such as "I loved this tool a lot until they turned it into worthless garbage of software". Study S59 posited that when faced with complex problems, novices who exhibit a more positive communication style are more likely to get assistance

Table 9
Types of tasks recommended in the existing systems.

Task type	Studies	Count
Repository to contribute	S5, S6, S7, S15, S16, S17, S18, S19, S20, S21, S22, S23, S24, S30, S32, S36, S47, S50, S51, S52, S53, S54, S55, S56, S61, S62, S63, S64, S65	27
Fixing issues	S1, S10, S11, S12, S34, S37, S41, S42, S43, S44, S45, S48	12
Testing tasks	S2, S31, S58, S60	4
Good First Issues	S9, S33, S35, S59	4
Question to answer	S25, S26, S38, S49	4

from expert developers. Hence, they utilize the titles and descriptions of previous Pull Requests and issues to compute sentiment measurements for newcomers. The measurements incorporate the textual sentiment polarity mean and median values of their previously submitted PRs and issues that have been reported.

Demographic information, such as *age, location, education, etc.*, are currently included in Studies S15 and S28, while Study S38 suggested considering the *geographic region* in the future. Study S15 modeled developer and project profiles by analyzing their basic information and mining user online behavior logs. Then they filtered out the projects based on the location, age, and education level requirements. This filtration process was done to speed up the recommendation process. It has been observed that workers from certain countries tend to prefer collaborating with task posters from specific other countries. For example, workers from India often prefer to work with task posters from the US and the UK [32]. Hence, Study S28 captured a worker's preference for collaborating with task posters from certain countries. They extracted the worker's location, education, etc., from the worker's profile information.

The developer's *discussion engagement level* was used in the recommendation system proposed in Study S12. It aimed to understand how developers contribute to communication. For example, some developers play a key role in discussions that may reflect their expertise or the importance of the topic, potentially correlating with specific API domains [33]. Study S1 suggested including the developer's level of engagement in discussions as a further improvement.

Study S26 considered *engagement states (anxiety, apathy, and boredom)* when recommending questions. Study S10 suggested considering *user satisfaction* as a future improvement to their recommendation system. Study S29 considered software practitioner's *motivation* when recommending tasks, while Studies S18 and S28 mentioned it as a future improvement. Studies S12, S28, and S29 suggested including the developer's *career goals* as a future improvement to the proposed systems. Study S10 suggested including *responsiveness*, which refers to how long it takes for the user to respond to the recommendation. The quicker the response is, the higher the responsiveness is.

RQ3 Summary: Human factors play a major role in software engineering. Hence, we analyzed the human factors currently being used in the existing recommendation systems and the human factors suggested to be included as future improvements to the systems platform (See Table 10). Some of the human factors currently being used in the existing recommendation systems are personality types, sentiments, motivation, and career goals.

3.5. RQ4: What platforms are currently being used to recommend crowdsourced software engineering tasks?

Table 11 shows the crowdsourcing platforms used in the selected primary studies.

GitHub is the most popular platform used in the primary studies. It is used in 38 out of 65 studies (58.4% of the studies). It is a developer-based platform to build, scale, and deliver secure software [34]. Some of the Github projects used in the existing studies are JabRef, PowerToys, and Audacity. A further analysis of these projects is available in Table 12.

Topcoder collaborates with clients to provide tailored solutions and support for both small tasks and large-scale projects throughout the entire software development process [35]. The Topcoder project categories used in the primary studies are BugBash, Code, UI, Prototype, and Quality Assurance.

Bugzilla is a comprehensive system for tracking defects or bugs in software. It helps developer teams manage and monitor issues, enhancements, and change requests in their products efficiently [36]. Eclipse, Mozilla, and LibreOffice are the projects used from the Bugzilla platform in the primary studies.

Stack Overflow is a website where programmers, both professionals and enthusiasts, ask and answer questions about programming. It aims to create a collection of thorough and top-notch answers for every programming-related question [37].

JointForce, created by ChinaSoft International, is a reliable IT service platform that uses social collaboration and a sharing economy model. It ranks 8th among China's top 100 software companies [38].

uTest is a platform where freelance software testers can learn, earn money, and contribute to the improvement of digital products for various brands [6].

Amazon Mechanical Turk (MTurk) is a crowdsourcing platform where people and businesses can hire remote workers to complete tasks and jobs online [39].

Testin is a platform that offers testing, security, promotion, optimization, and AI solutions for applications. It serves over one million developers and enterprises globally [40].

Baidu, Zhubajie, Oschina, Witkey, MoocTest, and Chinese open source crowdsourcing platform, are popular Chinese crowdsourcing platforms used in the primary studies.

Table 12 shows the scales of the projects available in GitHub used in the primary studies. The projects with less than 100 contributors were considered small-scale projects, projects with 100-500 contributors were considered medium-scale projects, and projects with more than 500 contributors were considered large-scale projects. There were 3 small-scale projects (15%), 6 medium-scale projects (30%), and 11 large-scale projects (55%). We could not gather information about the project scales of the other platforms as they were not accessible.

RQ4 Summary: We observed that GitHub is the most popular 58.4% crowdsourced software engineering platform among the selected primary studies. Further, we observed that the majority (55%) of the GitHub projects that were used to recommend tasks were large-scale projects with more than 500 contributors. Further, Bugzilla, Topcoder, and Utest are some other popular CSE platforms (see Table 11).

3.6. RQ5: Which features of platforms can be used to recommend crowdsourced software engineering tasks?

Features of the platform include structured elements such as issue or task descriptions, pull requests, comments, labels, etc.; these are

Table 10
Human factors included and suggested to be included in the studies.

Human Factor	Studies covering human factor	Studies suggesting human factor
Topic and task type preference	S2, S13, S18, S31, S54, S55, S57, S58, S60	S35
Skill preference	S1, S5, S15, S16, S32, S59, S64	N/A
Social and behavioral preferences	S8, S10, S43, S65	S5
Personality	S4, S57, S58	N/A
Sentiments	S9, S59	N/A
Age, location, education level, gender	S15, S28	S38, S49
Discussion engagement level	S12	S1
Engagement status (Anxiety, Apathy, and Boredom)	S26	N/A
Motivation	S29	S18, S28
Career goals	N/A	S12, S28, S29
User satisfaction	N/A	S10
Responsiveness	N/A	S10

tangible, system-generated artifacts within the development platform. Table 13 shows the features and the different attributes of the features of crowdsourcing platforms used to extract data.

Issue or task description is the most popular feature used to extract different attributes. It was used in 30 primary studies. Title and description, attached files or links to other related issues, and assignee were extracted from the issue or task description. For example, S1 extracted the title and body from the issue descriptions.

User activities such as starring, forking, watching, and bidding were extracted in 24 primary studies. This is the second most popular feature used in primary studies. For example, Study S7 extracted starring and forking repositories in the GitHub platform to identify user preferences.

Pull requests are the third most popular feature used to extract different attributes. It was used in 18 primary studies. Commit message text, names of the files changed in the pull request, and pull request comments were extracted from the pull requests. For example, Study S1 extracted the name of the files changed, and commit messages from the pull requests to identify the relevant domain for the issues.

Comments were used in 14 primary studies. commenter's name and the text were extracted from the comments. Study S12 extracted commenter names to get the total number of commenters which they used as a social metric in their recommendation system.

Labels and tags provide insightful data about the tasks. Hence, they were used in 13 studies. Issue types such as verified, fixed, invalid, duplicate, wontfix, worksforme, critical, difficulty levels such as good first issue, medium difficulty, etc., tagged keywords were extracted from labels and tags. For example, Study S2 used bug labels and duplicate labels to identify the bug status.

Code was used in 7 studies. APIs used in the code, variables, classes, identifiers, code snippets, and components or methods related to the tasks were extracted from the code. For example, Study S1 analyzed the code to identify the APIs used in the code as they recommend tasks based on App Programming Interface domains.

ReadME file was used to extract data in 5 studies. The purpose and the usage of the project, information about different methods used in the project, and the copyright information were extracted from the Readme files of projects. For example, Study S17 extracted the purpose, usage, and programming language used in the project from the ReadMe file.

Further, *Mailing lists, forums, source code management systems (SCM), and wikis* were also used to extract data in Study S41.

RQ5 Summary: Issue or task description is the most popular feature used to extract data in primary studies. Further, pull requests, comments, labels, tags, etc., are other popular features used to extract data (see Table 13). From those features, different attributes, such as commit message, task type, APIs used, etc, were extracted.

4. Threats to validity

Although we adhered closely to the recommendations outlined in [13], our SLR may have similar threats to validity as other SLRs in software engineering. The outcomes of our SLR might have been affected by the following validity threats.

Internal validity. To mitigate threats to internal validity, we adhered to a predefined SLR protocol. The search string was iteratively refined and tested before execution in order to optimize the results. Further, we developed our search string according to PICOC criteria, which enhances its comprehensiveness. To minimize bias in the study selection process, we followed a multi-step filtering process. The first two authors validated the inclusion and exclusion criteria on a small subset of primary studies before applying them broadly. Papers were screened in several rounds: initially based on titles, abstracts, and keywords followed by a brief reading, and finally, a detailed review. Any disagreements regarding inclusion were resolved through discussions between the first and second authors. For data extraction, we developed a structured data extraction form (see Table 7) to ensure consistency. Since the first author conducted most of the extraction, regular discussions were held with the second author to clarify uncertainties and resolve discrepancies. A subset of the extracted data was further verified by the second and third authors.

Construct validity. To reduce threats to construct validity, we searched across six major digital libraries and employed two complementary search strategies: automatic database searches and snowballing. The selected primary studies provide highly relevant solutions to recommend a CSE task to a software practitioner. To further refine our selection process, we conducted several rounds of discussions to improve the inclusion and exclusion criteria.

Conclusion validity. To minimize biases in data synthesis and analysis, we adopted a collaborative approach. All authors participated in multiple rounds of discussions to determine the most effective way to categorize and present findings. This iterative process helped reduce subjective interpretations and ensured that our conclusions were well-grounded in the extracted data.

External validity. External validity reflects the generalizability of our findings. A potential limitation is the scope of the literature included. Our study relied on papers retrieved from six major digital libraries, including Scopus, which is known for its extensive indexing capabilities. However, studies outside these databases may not have been captured. To mitigate this, we employed snowballing techniques, manually reviewing references from selected papers to identify additional relevant studies. Further, we did not impose any time limitation on our search to capture all developments in the area. To improve the generalizability of our findings, future work could incorporate additional sources.

5. Related work

During this review, we only considered peer-reviewed and published work and excluded blogs and media articles. Hence, we found

Table 11
Crowdsourcing platforms and projects.

Platform	Projects	Studies
Github	JabRef	S1, S12
	PowerToys	S1, S12
	Audacity	S1, S12
	Qt	S9
	Eclipse	S9, S11, S43
	LibreOffice	S9
	Mozilla	S11, S43
	CNCF projects	S16
	babel	S33
	Bitcoin	S33
	Jest	S33
	Lighthouse	S33
	Scaffold	S33
	Packer	S33
	Graphql-engine	S33
	Minikube	S33
	React-admin S33	S33
	React-server S33	S33
	Python scientific computing software eco system	S34
	GH Archive	S21
	Not specified	S5, S6, S7, S10, S13, S17, S18, S19, S20, S22, S23, S24, S30, S32, S35, S36, S47, S50, S51, S52, S53, S54, S55, S56, S59, S62, S63, S64, S65
Topcoder	BugBash	S27
	Code	S27
	UI	S27
	Prototype	S27
	Quality Assurance	S27
	Not specified	S28, S29, S39
Bugzilla	Eclipse	S3
	Mozilla	S3
	LibreOffice	S3
	Not specified	S10, S37
Stackoverflow	Not applicable	S25, S26, S38, S49
Jointforce	Not specified	S8, S15, S61
Baidu CrowdTest crowdtesting platform	Not specified	S2, S31
Utest	Not specified	S31
Amazon Mechanical Turk (MTurk)	Not specified	S40
Testin	Not specified	S31
Zhubajie	Not specified	S14
Oschina	Not specified	S14
Witkey	Not specified	S14
MoocTest	Not specified	S31
Chinese open source crowdsourcing platform	H5 applications, WeChat applications	S46

one related existing systematic literature review and one survey study. We summarize the key aspects of these studies below.

Zhen et al. [12] conducted an SLR that focuses on the software tasks in the crowdsourcing domain at a high level, along with the tasks in non-software engineering domains such as data entry, writing, editing, etc. The study examined the impact of task assignment in crowdsourcing, including the consequences of assigning a task to an appropriate worker and an inappropriate worker. The study highlights that improper task-worker matching not only affects result quality but also wastes valuable resources such as time, money, and client trust, underscoring the need for an optimal task assignment model. It emphasized the application of crowdsourcing in software engineering and analyzed existing methods for effective task allocation. Further, it identified 52 existing crowdsourcing platforms such as Amazon MechanicalTurk (AMT),[4] Topcoder,[5] CastingWords.[6] Our SLR provides a comprehensive, in-depth analysis of personalized task recommendations to software practitioners in crowdsourcing platforms, focusing on recommending a task to a particular software practitioner in a crowdsourcing platform. Further, the existing SLR is conducted on the

[4] https://www.mturk.com/
[5] https://www.topcoder.com
[6] http://castingwords.com/

Table 12
GitHub project scales.

Github Project	Link	Contributors count	Scale
Python scientific computing software ecosystem	https://github.com/python/cpython	2619 (for the repository with the highest number of stars)	large
Jest	https://github.com/jestjs/jest	1525	large
LibreOffice	https://github.com/LibreOffice/core	1251	large
Packer	https://github.com/hashicorp/packer	1202	large
babel	https://github.com/babel/babel	1071	large
Bitcoin	https://github.com/bitcoin/bitcoin	938	large
Minikube	https://github.com/kubernetes/minikube	861	large
CNCF projects	https://github.com/cncf/landscape	794 (for the repository with the highest number of stars)	large
JabRef	https://github.com/JabRef/jabref	614	large
React-admin	https://github.com/marmelab/react-admin	580	large
Graphql-engine	https://github.com/hasura/graphql-engine	546	large
PowerToys	https://github.com/microsoft/PowerToys	433	medium
Mozilla	https://github.com/mozilla/pdf.js	374 (for the repository with the highest number of stars)	medium
Lighthouse	https://github.com/GoogleChrome/lighthouse	332	medium
Qt	https://github.com/qt/qt	231	medium
Audacity	https://github.com/audacity/audacity	203	medium
Eclipse	https://github.com/eclipse/mosquitto	131 (for the repository with the highest number of stars)	medium
Scaffold	https://github.com/scaffold-eth/scaffold-eth	94	small
React-server	https://github.com/redfin/react-server	72	small
GH Archive	https://github.com/igrigorik/gharchive.org	53	small

Table 13
Platform features and attributes used to extract data.

Feature	Attributes	Studies
Issue/task description	Title and description	S1, S3, S5, S7, S9, S11, S12, S14, S15, S27, S28, S30, S33, S34, S35, S37, S38, S39, S41, S42, S43, S44, S45, S46, S49, S54, S55, S59, S63
	Attached files or links	S3
	Assignee	S3, S48
User activities	Fork, star, watch, bid, etc	S5, S7, S16, S18, S19, S20, S21, S22, S23, S24, S32, S34, S35, S36, S44, S47, S50, S51, S56, S57, S62, S63, S64,S65%
Pull Requests	Commit messages	S1, S10, S11, S12, S18, S20, S33, S36, S41, S44, S53, S59, S65
	Names of the files changed	S1, S12
	Pull request review comment	S2, S16, S21, S43, S51
Comments	Commenter name	S12
	Comment text	S1, S6, S10, S12, S13, S21, S33, S35, S36, S41, S43, S44, S64, S65
Labels, tags	Issue or task type	S2, S3, S34, S37, S40, S41, S42, S43, S48, S51, S59
	Tagged keywords	S25, S61
Code	APIs used	S1
	Variables, classes, identifiers, code snippets	S13, S33, S34, S59
	Related components or methods	S17, S48
Readme file	Purpose and usage, methods information, copyright information	S17, S50, S52, S56, S59
Mailing lists, forums, source code management systems (SCM) and wikis.	N/A	S41

studies published from 2010 to 2019, whereas our SLR has no time duration restriction. Our search string significantly differs from the one used in [12]. They identified 120 relevant studies and selected 70 most relevant studies for their SLR. Only one primary study overlaps with the selected primary studies of our SLR. Regarding the research questions, our RQ1, RQ2, and RQ4 partially overlap with the existing literature review's RQ3, RQ2, and RQ4, respectively. Moreover, we focus on assigning a suitable task to a particular software practitioner, whereas

the majority of primary studies in the existing SLR focus on assigning a practitioner to a task. Since the existing SLR is a high-level study on the software engineering domain, the results include those unrelated to the software engineering field. Hence, there is a significant difference between our SLR and the existing SLR.

Mao et al. [1] performed a survey that focuses on how crowdsourcing is used in different stages of the software development life cycle along with its taxonomy. Further, it provides definitions for crowdsourced software engineering, crowdsourcing practices in software engineering, commercial platforms, and relevant case studies. The study highlighted that while many previous studies have cited benefits such as low cost and quick delivery, there is a lack of comparative research that validates these advantages against traditional development methods. It also highlighted the significance of quality control in crowdsourced projects. Further, the findings of the study emphasize the challenges of managing communication, expectations, and workflows among crowd contributors with varying skill levels and motivations. It also highlights concerns about intellectual property rights and the need for clear guidelines to protect both requesters and contributors. Additionally, the study notes that cultural and regional differences can affect collaboration, making it crucial to understand these differences for effective global crowd management and alignment with project goals. However, the survey is not based on any research questions as it is a broad analysis. They conducted the survey on the studies published from January 2006 to March 2015, whereas our SLR has no time duration restriction. They surveyed 210 publications out of 476 studies identified during the initial stage. There are no overlapping primary studies with our selected primary studies set for this SLR. Our search string significantly differs from the search string used in this survey. RQ4 in our SLR overlaps with the section "4.1 commercial platforms" in the survey. However, our SLR is focused on recommending crowdsourced software engineering tasks to software practitioners, whereas this survey is a broad analysis of crowdsourced software engineering. Hence, there is a significant difference in this survey with our SLR.

6. Discussion and future research directions

The research on task recommendation in Crowdsourced Software Engineering (CSE) is gaining increasing attention within the software engineering community. This is evidenced by a consistent upward trend in the volume of papers dedicated to this domain in the past decade (see Fig. 4). Notably, over half of the papers reviewed (32 papers, 50%) were published within the last three years. Several categories of software tasks have been recommended in several crowdsourcing platforms like GitHub and Topcoder using a large number of features and attributes in those platforms. It is crucial to systematically review and comprehensively document the various software task recommendation methods in crowdsourcing platforms to help understand the current progress of the area and potential future directions. Based on the insights of this SLR, along with the identified limitations, we have pinpointed several key challenges in the domain of recommending tasks to software practitioners on crowdsourcing platforms. We outline these challenges below as recommendations to the Software Engineering research community for conducting further research on this domain.

- **Human factors in software task recommendation systems**:
  Software is developed by humans, read by humans, and maintained by humans [41]. Human factors play a crucial role in software development [42]. Human factors affect software development team members, their activities, and ultimately, the quality of the product [15]. Ockiya and Lock [43] discovered a connection between human factors in remote software teams and their success or failure. Gaining a deeper understanding of this relationship and managing it effectively could help alleviate the challenges faced by remote teams. Some major challenges in CSE include ensuring effective communication and coordination, handling knowledge sharing, fostering motivation, building a network of volunteers, establishing trust, and facilitating team growth [4,44,45].
  Human factors have been integrated into some existing primary studies, and several studies plan to incorporate them as future enhancements to their recommendation systems. We analyzed the human factors currently being used or suggested to be used in the existing systems in Table 10. For example, Studies S9 and S59 used software practitioners' sentiment data in their recommendation system. Study S29 considers software practitioners' motivation when recommending tasks, whereas Studies S18 and S28 plan to consider motivation as a future improvement. Studies S12, S28, and S29 mentioned considering software practitioners' career aspirations in their recommendation systems as intended future improvements. Study S38 suggests including geographic regions, while Study S49 suggests demographic information. Hence, this reflects that there is an interest among researchers to explore human factors affecting their recommendation systems.
  Further, there are numerous ongoing studies on human factors in the software engineering domain. Researchers could significantly enhance their recommendation systems by incorporating these findings. For example, Dutra et al. identified 101 human factors that impact software development activities from various perspectives [15]. Further, Ortu et al. explored the correlation between the attractiveness of a project and the level of politeness exhibited by developers involved in its development [46]. They found that the more polite developers were, the more eager new developers were to collaborate on a project, and the longer they were willing to work on the project [47]. Guzman et al. found that the commit comments written on Mondays tend to contain more negative emotions than positives [48].
  Considering such human factors when recommending the tasks will improve the productivity of the practitioners [48]. Additionally, promoting team diversity in terms of age, gender, disability, or ethnicity influences its positive impact on team member innovation [15]. Hence, these factors can be considered when onboarding newcomers to projects to maintain team diversity. Dutra et al. further mention that considering human factors helps in lowering the churn rate of software practitioners after onboarding to a project [15].
  However, the main challenge of incorporating human factors into existing recommendation systems is the lack of data to extract human factors. Studies S4 and S57 tried, to some extent, to bridge this gap by introducing a new web platform to gather software practitioners' personality data. Further, gathering demographic information of software practitioners, such as age, gender, etc, may raise privacy and ethical concerns. For example, Study S4 mentioned that none of the vendors were willing to provide them with information regarding their clients' or developers' details. Privacy and ethical concerns could be the reason behind this situation. Hence, collecting such demographic information should be done with necessary careful handling.
  It is clear that different human factors that are still undetermined could be used to recommend software tasks in crowdsourcing platforms. Hence, we encourage researchers to do an in-depth investigation on integrating human factors into the existing recommendation systems and the potential challenges that may stem from this integration.
- **Transferring knowledge from one platform to another:**
  As new software crowdsourcing platforms often lack sufficient historical data on developer behavior, existing recommendation algorithms struggle to effectively match developers with new tasks [49]. Among the existing algorithms, collaborative filtering (CF) has gained significant recognition due to its versatility, interpretability, and independence from content-based features.

However, it is also highly susceptible to data sparsity and the cold start problem. To address these challenges, CF algorithms with transfer learning have received considerable attention [50,51]. By leveraging knowledge from data-rich auxiliary domains and transferring it to target domains with limited data, these systems help mitigate data sparsity issues and enhance recommendation accuracy. This transfer of knowledge enables new platforms to leverage existing user preferences, task information, and interaction patterns, thus enhancing their overall performance and user experience [52–54]. Given that contributors in CSE come from diverse backgrounds with varying levels of experience, leveraging prior data from different platforms can help match developers to projects more effectively.

For example, Studies S14, S15, and S27 mentioned addressing the "cold start" problem by transferring knowledge or merging more information from other platforms as future improvements to their systems. Researchers in other domains have researched transferring knowledge from other platforms for their recommendation systems. For example, Yan et al. investigated cross-platform social relationships and behavior information to tackle the cold-start problem in friend recommendation [55]. Specifically, they performed comprehensive data analysis to determine which information is most effectively transferable between platforms [55]. Pan et al. addressed the data sparsity issue in the recommendation systems by transferring knowledge about both users and items from auxiliary data sources [56]. Li et al. addressed the sparsity problem in individual rating matrices by learning a generative model from multiple related recommender systems [57].

Transferring knowledge can help initialize the recommendation engine and make informed recommendations even in the absence of historical data specific to the new platform [52]. Existing data from one platform can be used to bootstrap the recommendation system on another platform as it may reduce the need for collecting large amounts of new data [54]. Further, utilizing user preferences and interaction patterns from other platforms may allow for more personalized and relevant recommendations, enhancing user satisfaction [58].

However, sharing user data across platforms may raise privacy concerns [59]. Data breaches or unauthorized access during the transfer process can compromise sensitive information. Hence, careful consideration of data security measures and compliance with regulations is required. Further, it can be a technically complex task as it requires compatibility between different platforms, data formats, and systems [52,60,61].

It can be seen that only 4.7% of primary studies have identified this area for improvement, and it is still largely unexplored. There is an opportunity for more studies to explore transferring knowledge from one platform to another. Hence, we recommend that researchers consider integrating knowledge transferring from other platforms into their recommendation systems.

- **Cross-platform recommendation systems:**

Cross-platform recommendation systems provide users with personalized recommendations regardless of the platform they are signed up with.These systems enable a broader reach and cater to the diverse needs of users who participate in crowdsourcing platforms. Only 4.6% of the primary studies were tested on multiple crowdsourcing platforms. When the recommendation systems are designed to cater to a specific platform, those systems may be constrained by the features and capabilities of the platform they are built for [62,63]. It may limit their flexibility and adaptability. Further platform-specific recommendation systems may contribute to user lock-in where users may feel constrained to stay on a single platform due to a lack of integration with other services or options. For example, Xu et al. [64] proposed a cross-platform developer recommendation algorithm based on feature matching. Experimental results demonstrate that the proposed algorithm outperforms existing recommendation algorithms across various evaluation metrics.

The flexibility of platform-independent solutions enables the recommendation system to cater to diverse user needs and preferences without being tied to a specific platform's constraints. The interoperability of these solutions facilitates collaboration and data exchange between platforms, enabling more comprehensive and holistic recommendation strategies [65]. This is particularly important in crowdsourcing, where users often contribute to projects across different platforms and bring a variety of skills, motivations, and preferences to the table.

However, developing cross-platform recommendation systems could be challenging as some required data may not be available on every platform in the same form [63,66]. For example, the Topcoder platform offers money as a prize to the winners of their tasks, whereas GitHub does not provide such prizes for every task [34,35]. Hence, considering offered developer incentives as a recommendation factor in a cross-platform recommendation system is a complex task. On the other hand, when extending recommendation systems to work across multiple platforms, there is a risk of sacrificing the unique features of each platform, which could offer valuable insights for generating recommendations. For example, Studies S7 and S18 used the "GitHub's forking a repository" feature to extract user preferences. When extending those recommendation systems to cater to multiple platforms, they may not use such unique platform features to extract data.

We encourage the researchers to build cross-platform recommendation systems to recommend software tasks to software practitioners in crowdsourcing platforms.

- **Comprehensive evaluation, standardized evaluation metrics, and benchmarking are required in future studies:**

Evaluation of the existing systems often lack comprehensiveness due to the narrow focus on technical metrics, limited data sets, limited real-world studies, etc. Further evaluations covering more aspects of the proposed recommendation systems were suggested in the primary Studies S2, S11, S13, S18, S20, S23, S34, S37, S48, S50, and S62. Study S1 assesses the introduced tool's impact where they gathered feedback from contributors and analyzed how it influenced their choices through controlled experiments. Studies S11, S13, S18, S48, and S50 plan to broaden experiments to encompass more datasets, aiming to mitigate potential threats to external validity. Study S20 plans to conduct surveys involving junior and senior developers to ascertain the effectiveness of the introduced metrics, especially those related to commenting activities, and apply them to solve various problems. Study S23 intends to perform survey-based online experiments with actual developers to assess the usefulness of the proposed recommendation system. Study S34 plans to evaluate the recommendation model on larger projects in future research, including both open-source and commercial projects, to ensure a comprehensive evaluation. Study S37 plans to conduct user studies to evaluate the usability and effectiveness of Tesseract, particularly focusing on its search features. Study S62 aims to apply their proposed model to experiment on larger and richer datasets to validate its performance.

These further evaluations help to understand current gaps in the existing literature. Establishing standardized evaluation metrics, datasets, and benchmarking frameworks for comparing and assessing the performance of different recommendation approaches could facilitate reproducibility, transparency, and rigor in evaluating the effectiveness of software task recommendation systems across various settings and domains [67]. Further evaluations covering multiple aspects of recommendation systems, including technical metrics, real-world studies, and user feedback, provide

more comprehensive recommendations [68]. Evaluating recommendation models on larger and richer datasets helps validate their performance in real-world settings, ensuring that they can scale effectively and provide accurate recommendations across different contexts [68,69].

However, conducting comprehensive evaluations, including user studies, surveys, and experiments on larger datasets, can be resource-intensive in terms of time, effort, and cost [68].

We recommend researchers do comprehensive evaluations when developing recommendation systems, and community to establish standardized evaluation metrics and benchmarking.

- **Integrating recommendation systems with development workflow tools:**

Professional software developers dedicate about one-third of their time to working within an integrated development environment (IDE), making it the most frequently used application throughout their workday [70]. Hence, numerous studies have been conducted to enhance these IDEs for a seamless development experience. For example, Bacchelli et al. introduced an Eclipse plugin for integrating Stack Overflow community knowledge directly into the IDE [71]. Luca Ponzanelli developed an Eclipse IDE plugin that automatically fetches, assesses, and recommends Stack Overflow discussions to developers once a specific confidence threshold is exceeded [72].

Integrating recommendation systems with development workflow tools involves embedding recommendation functionalities directly into software development tools and environments commonly used by developers, such as IDEs, version control systems, project management tools, or communication platforms. For example, Study S1 suggests integrating the proposed recommendation system with Github actions or bots. Study S3 developed an open source plugin for some of JetBrains' open source IDEs such as IntelliJ IDEA, CLion, etc.

Integrating the recommendation system with development workflow tools streamlines the task discovery and assignment process, enabling developers to access relevant tasks directly within their existing development environment without needing to switch between different tools or platforms. Further, current adaptive IDE systems, such as the one proposed by Schmidmaier et al. [73], can track the task context, developer's expertise level, and interaction patterns directly from the IDE. In crowdsourced development, contributors are diverse and work remotely. Unlike traditional teams, where roles and expertise are known, crowdsourcing platforms lack direct visibility into a developer's real-time interests and skill level. By leveraging real-time information from IDEs, recommendation systems can refine their recommendations to better align with the developer's context. For instance, if the IDE detects that a developer is working on a machine learning project and has an advanced expertise level, the system can prioritize recommending complex ML-related crowdsourced tasks.

However, different development tools and environments have varied architectures and APIs that make the integration process complex [74]. For example, integrating with Eclipse IDE might differ significantly from integrating with Visual Studio IDE [75]. This can be addressed by utilizing a plugin-based architecture where the recommendation functionalities are provided as plugins or extensions for different IDEs and tools [75,76]. Further, maintaining compatibility with continuously evolving tools and the latest versions of the IDEs can be challenging. Another key challenge of integrating recommendation systems with development workflow tools is that recommendation systems can consume significant computational resources, potentially slowing down the development environment [77].

We suggest researchers integrate their recommendation systems with development workflow tools.

- **The need for interactive recommendation systems:**

Interactive recommendation systems enable users to actively engage with the recommendation process by providing feedback, preferences, and additional input [78]. In the context of CSE, where contributors vary in expertise, availability, and motivations, interactive recommendations can help match developers with tasks more effectively. Tailoring recommendations to individual preferences increases the relevancy of the recommendations. Continuous feedback may help refine and improve the recommendation algorithms over time. Hence, such interaction enhances user satisfaction and improves the user experience by adapting to changing preferences over time [79,80]. For example, Study S3 currently allows users (specifically Eclipse developers) to provide feedback through likes, dislikes, or snoozes (reminders) for issues. They utilize this feedback to hide certain issues in case of dislikes and to re-rank the recommendation list based on likes. In the future, they intend to leverage this data to automatically enhance and adjust the recommendation model.

Hau Xuan Pham & Jason J. Jung developed an interactive recommendation system designed to assist users in rectifying their own ratings. Their approach involved determining whether user ratings align with their preferences, and subsequently correcting these ratings to enhance recommendations [81]. Mahmood et al. introduced a novel recommender system that autonomously learns an adaptive interaction strategy to aid users in achieving their interaction objectives [82]. Hariri et al. introduced an interactive recommender system that can identify and adjust to changes in context by analyzing the user's ongoing behavior [83]. Additionally, the development of Large Language Models (LLMs), like ChatGPT and GPT-4, has transformed the fields of Natural Language Processing (NLP) and Artificial Intelligence (AI). These models exhibit exceptional proficiency in fundamental tasks of language understanding and generation, alongside notable generalization abilities and reasoning skills [84]. Study S64 mentioned in the future works, integrating LLMs like GPT with their recommendation system can significantly improve recommendation accuracy by capturing complex linguistic patterns and semantics. LLMs could assist in dynamically refining task recommendations by processing natural language queries, extracting intent from developer interactions, and suggesting relevant tasks that align with evolving skills and interests. Hence, LLMs can be employed to provide interactive recommendations as those excel at processing and understanding natural language input and leverage context from user queries and previous interactions to generate more relevant and personalized recommendations [85]. LLMs can help address the cold-start problem by leveraging limited data to generate personalized, context-aware recommendations. Their ability to process customizable prompts in chat-based systems enhances user engagement. For example, the LLM-based recommendation system in [86] allows users to express their preferences and intents in natural language. This approach outperforms traditional methods relying solely on user-item interactions. Given their effectiveness in tackling data sparsity and improving efficiency, language models have become a promising approach in advancing recommendation systems in both academia and industry [79].

However, implementing interactive features can add complexity to the system and increase computational overhead [82]. Further, interactive recommendation systems require collecting vast amounts of user data, including preferences, feedback, and interaction history. The extent and granularity of this data can raise concerns about user privacy [80]. Hence, we suggest developing interactive recommendation systems for CSE task recommendation in the future.

A summary of future research directions and the potential challenges of those improvements are available in Table 14.

Table 14
Future research directions and potential challenges.

Future Research Direction	Potential Challenges
Integrating human factors into recommendation systems	Lack of data to extract human factors. Privacy and ethical concerns.
Transferring knowledge from one platform to another	Privacy concerns. Data breaches or unauthorized access during the transfer process. Technical complexity.
Introducing cross-platform recommendation systems	May not be able to use unique platform features.
Comprehensive evaluation, standardized evaluation metrics, and benchmarking are required in future studies	Can be resource-intensive in terms of time, effort, and cost.
Integrating recommendation systems with development workflow tools	Technical complexity. Maintaining compatibility with continuously evolving tools and the latest versions. May consume significant computational resources.
Developing interactive recommendation systems	May add complexity to the system and increase computational overhead. Privacy concerns.

Table 15
Selected primary studies.

Ref	ID	Title	Venue	Year
[87]	S1	GiveMeLabeledIssues: An Open Source Issue Recommendation System	International Conference on Mining Software Repositories (MSR)	2023
[88]	S2	Context-Aware Personalized Crowdtesting Task Recommendation	IEEE Transactions On Software Engineering	2022
[89]	S3	Towards Issue Recommendation for Open Source Communities	International Conference on Web Intelligence	2019
[24]	S4	Impact of Personality on Task Selection in Crowdsourcing Software Development: A Sorting Approach	IEEE Access	2017
[90]	S5	Personalized Repository Recommendation Service for Developers with Multi-modal Features Learning	International Conference on Web Services (ICWS)	2023
[91]	S6	Project Recommendation for Open Source Communities	International Conference on Emerging Research in Electronics, Computer Science and Technology (ICERECT)	2022
[92]	S7	REPERSP: Recommending Personalized Software Projects on GitHub	International Conference on Software Maintenance and Evolution	2017
[93]	S8	Task Recommendation with Developer Social Network in Software Crowdsourcing	Asia-Pacific Software Engineering Conference	2016
[94]	S9	A Simple NLP-Based Approach to Support Onboarding and Retention in Open Source Communities	International Conference on Software Maintenance and Evolution	2018
[95]	S10	A Recommendation Method for Social Collaboration Tasks Based on Personal Social Preferences	IEEE Access	2018

(*continued on next page*)

7. Conclusion

This SLR has provided a detailed analysis of software task recommendations to software practitioners in crowdsourcing platforms. Our analysis equips researchers with a comprehensive understanding of existing research in the domain, including the existing methods, benefits, and limitations of those methods, different task types recommended, use of human factors in recommendations, popular crowdsourcing platforms, and the features of those platforms used to extract data. Further, we highlighted the key areas for future research. For the SLR, We adhered to the systematic process outlined by Kitchenham et al. [13]. We selected highly relevant primary studies for the SLR after a rigorous filtration process. Our main findings indicate that the number of studies on crowdsourced software engineering task recommendations significantly increased over the past couple of years, and we anticipate that this trend will continue. The identified future directions for further research in this domain include: (1) More studies are needed to explore

Table 15 (*continued*).

Ref	ID	Title	Venue	Year
[96]	S11	Combining Word Embedding with Information Retrieval to Recommend Similar Bug Reports	International Symposium on Software Reliability Engineering	2016
[33]	S12	Tell Me Who Are You Talking to and I Will Tell You What Issues Need Your Skills	International Conference on Mining Software Repositories (MSR)	2023
[97]	S13	Semantic Similarity-Based Contributable Task Identification for New Participating Developers	Journal of Information and Communication Convergence Engineering	2018
[98]	S14	Cold-Start Developer Recommendation in Software Crowdsourcing: A Topic Sampling Approach	International Conference on Software Engineering and Knowledge Engineering	2017
[99]	S15	A Reinforcement Learning Solution to Cold-Start Problem in Software Crowdsourcing Recommendations	International Conference on Progress in Informatics and Computing (PIC)	2018
[100]	S16	Open Source Software Supply Chain Recommendation Based on Heterogeneous Information Network	International Symposium on Benchmarking, Measuring and Optimization	2023
[101]	S17	Graph Convolutional Network-Based Repository Recommendation System	Computer Modeling in Engineering and Sciences	2023
[102]	S18	Recommendation System for Open Source Projects for Minimizing Abandonment	The International FLAIRS Conference Proceedings	2022
[103]	S19	Open-source project recommendation model	E3S Web of Conferences	2021
[104]	S20	New developer metrics for open source software development challenges: An empirical study of project recommendation systems	Applied Sciences	2021
[105]	S21	A GitHub Project Recommendation Model Based on Self-Attention Sequence	International Conference on Big Data Engineering	2021
[106]	S22	Improving Personalized Project Recommendation on GitHub Based on Deep Matrix Factorization	International Conference on Collaborative Computing: Networking, Applications and Worksharing	2021
[107]	S23	Personalized project recommendation on GitHub	Science China Information Sciences	2018
[108]	S24	Scalable relevant project recommendation on GitHub	Asia-Pacific Symposium on Internetware	2017
[109]	S25	Suggesting Questions that Match Each User's Expertise in Community Question and Answering Services	International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)	2019
[110]	S26	HRCR: Hidden Markov-Based Reinforcement to Reduce Churn in Question Answering Forums	Pacific Rim International Conference on Artificial Intelligences	2019
[111]	S27	TDMatcher: A topic-based approach to task-developer matching with predictive intelligence for recommendation	Applied Soft Computing	2021
[32]	S28	TasRec: A Framework for Task Recommendation in Crowdsourcing	International Conference on Global Software Engineering	2022

(*continued on next page*)

the effect of different human factors such as the personality of software practitioners, motivation, and career goals on CSE; (2) limited attention has been paid to transferring knowledge from other related platforms, which can be beneficial in addressing the "cold start" problem; (3) Investigation is needed to introduce generalizable platform-independent recommendation systems; (4) Need to conduct comprehensive further evaluation on existing systems and need establish standardized evaluation metrics and bench-marking; (5) Investigation needed to integrate the recommendation system with development workflow tools; (6) Can develop existing systems to interactive recommendation systems with

Table 15 (*continued*).

Ref	ID	Title	Venue	Year
[112]	S29	CrowdAssist: A multidimensional decision support system for crowd workers	Journal of Software: Evolution and Process	2021
[113]	S30	ReBOC: Recommending Bespoke Open Source Software Projects to Contributors	Symposium on Visual Languages and Human-Centric Computing (VL/HCC)	2022
[114]	S31	Leveraging Android Automated Testing to Assist Crowdsourced Testing	IEEE Transactions on Software Engineering	2023
[115]	S32	Open Source Repository Recommendation in Social Coding	International ACM SIGIR Conference on Research and Development in Information Retrieval	2017
[116]	S33	Characterizing and predicting good first issues	International Symposium on Empirical Software Engineering and Measurement (ESEM)	2021
[117]	S34	Effective Recommendation of Cross-Project Correlated Issues based on Issue Metrics	Asia-Pacific Symposium on Internetware	2023
[118]	S35	Recommending good first issues in GitHub OSS projects	International Conference on Software Engineering (ICSE)	2022
[11]	S36	RepoLike: personal repositories recommendation in social coding communities	Asia-Pacific Symposium on Internetware	2016
[119]	S37	Which bug should I fix: helping new developers onboard a new project	International Workshop on Cooperative and Human Aspects of Software Engineering	2011
[120]	S38	User intimacy model for question recommendation in community question answering	Knowledge-Based Systems	2020
[121]	S39	Learn or earn? Intelligent task recommendations for competitive crowdsourced software development	Hawaii International Conference on System Sciences	2018
[122]	S40	Probabilistic matrix factorization with active learning for quality assurance in crowdsourcing systems	International Conference WWW/Internet	2015
[123]	S41	Semantic tools for improving software development in open source communities	International Semantic Web Conference	2013
[124]	S42	Graph collaborative filtering-based bug triaging	The Journal of Systems and Software	2023
[125]	S43	A spatial–temporal graph neural network framework for automated software bug triaging	Knowledge-Based Systems	2022
[126]	S44	GFI-Bot: Automated Good First Issue Recommendation on GitHub	ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering	2022
[127]	S45	RAPTOR: Release-Aware and Prioritized Bug-Fixing Task Assignment Optimization	International Conference on Software Maintenance and Evolution (ICSME)	2019
[128]	S46	Software Crowdsourcing Task Allocation Algorithm Based on Dynamic Utility	IEEE Access	2019
[129]	S47	Recommending Relevant Projects via User Behaviour: An Exploratory Study on Github	International Workshop on Crowd-based Software Development Methods and Technologies	2014

(*continued on next page*)

Table 15 (*continued*).

Ref	ID	Title	Venue	Year
[130]	S48	Developer load balancing bug triage: Developed load balance	Expert Systems	2022
[131]	S49	Tracking user-role evolution via topic modeling in community question answering	Information Processing and Management	2019
[132]	S50	Detecting Similar Repositories on GitHub	International Conference on Software Analysis, Evolution and Reengineering (SANER)	2017
[133]	S51	Recommending Relevant Open Source Projects on GitHub using a Collaborative-Filtering Technique	International Journal of Open Source Software and Processes	2015
[134]	S52	open source software recommendations using github	International Conference on Theory and Practice of Digital Libraries	2018
[135]	S53	Recommending GitHub Projects for Developer Onboarding	IEEE Access	2018
[136]	S54	Ghtrec: A personalized service to recommend github trending repositories for developers	International Conference on Web Services (ICWS)	2021
[137]	S55	Sequential recommendations on github repository	Applied Sciences	2021
[138]	S56	CrossSim: exploiting mutual relationships to detect similar OSS projects	Euromicro Conference on Software Engineering and Advanced Applications	2018
[25]	S57	Task Assignment and Personality: Crowdsourcing Software Development	Research Anthology on Agile Software, Software Development	2022
[31]	S58	To enhance effectiveness of crowdsource software testing by applying personality types	International Conference on Software and Information Engineering	2019
[139]	S59	Personalized First Issue Recommender for Newcomers in Open Source Projects	International Conference on Automated Software Engineering (ASE)	2023
[140]	S60	Crowdsourced Testing Task Assignment based on Knowledge Graphs	International Conference on Software Quality, Reliability and Security (QRS)	2022
[52]	S61	Transfer learning for cross-platform software crowdsourcing recommendation	Asia-Pacific Software Engineering Conference (APSEC)	2017
[141]	S62	Graph contextualized self-attention network for software service sequential recommendation	Future Generation Computer Systems	2023
[142]	S63	Relevant Projectrecommendation System Using Developer Behavior And Project Features	International Journal of Advanced Trends in Computer Science and Engineering	2021
[143]	S64	CodeCompass: NLP-Driven Navigation to Optimal Repositories	International Conference on Pervasive Computing and Social Networking	2024
[144]	S65	Multi-objective optimization and integrated indicator-driven two-stage project recommendation in time-dependent software ecosystem	Information and Software Technology	2024

the help of large language models which enable users to actively engage with the recommendation process.

CRediT authorship contribution statement

Shashiwadana Nirmani: Writing – original draft, Visualization, Software, Methodology, Investigation, Data curation, Conceptualization. **Mojtaba Shahin:** Writing – review & editing, Supervision. **Hourieh Khalajzadeh:** Writing – review & editing, Supervision. **Xiao Liu:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Deakin University Postgraduate Research Scholarship (DUPR-ROUND 0000018830).

Table 16
Quality assessment scores for primary studies.

ID	QA1	QA2	QA3	QA4	QA5	ID	QA1	QA2	QA3	QA4	QA5
S1	5	5	4	5	2	S34	4	3	3	3	3
S2	5	4	4	5	4	S35	4	3	2	4	2
S3	4	5	3	3	3	S36	3	3	2	3	2
S4	3	2	2	3	1	S37	3	3	3	1	1
S5	3	3	4	4	2	S38	3	3	3	3	3
S6	3	2	3	4	2	S39	4	4	3	3	3
S7	5	3	2	3	1	S40	3	2	4	3	2
S8	4	2	4	4	2	S41	3	1	2	1	1
S9	4	3	4	4	4	S42	4	3	3	3	4
S10	3	2	3	3	2	S43	4	2	4	4	3
S11	3	4	3	4	4	S44	3	4	3	3	2
S12	4	3	3	4	4	S45	4	2	3	1	1
S13	3	2	3	3	3	S46	3	3	4	4	2
S14	3	3	4	4	2	S47	4	5	2	3	1
S15	4	4	4	3	3	S48	4	4	3	4	3
S16	3	2	3	3	2	S49	4	3	3	3	1
S17	3	3	3	4	3	S50	3	3	4	4	4
S18	3	3	2	3	1	S51	4	3	3	3	2
S19	2	3	1	3	1	S52	2	4	3	1	1
S20	4	4	3	3	3	S53	4	3	4	4	5
S21	4	3	2	3	2	S54	3	4	3	4	2
S22	3	4	3	4	4	S55	3	4	3	3	2
S23	4	3	3	4	4	S56	4	3	3	3	3
S24	4	3	3	4	3	S57	3	2	3	3	1
S25	3	3	3	3	3	S58	2	2	2	3	1
S26	2	3	3	1	1	S59	4	3	3	4	3
S27	4	3	4	4	2	S60	3	3	3	3	1
S28	3	3	5	3		S61	4	3	4	3	1
S29	3	3	4	4	2	S62	3	3	3	4	1
S30	3	3	2	2	2	S63	3	3	2	2	1
S31	3	3	4	4	4	S64	3	3	3	3	2
S32	3	4	2	3	1	S65	3	3	4	4	1
S33	4	4	3	3	3						

Appendix A. Primary studies

See Table 15

Appendix B. Quality assesment scores

See Table 16

Data availability

The SLR data is available at https://github.com/shashiwadana/CSE-TASK-REC-DATA

CSE-TASK-REC-DATA (Original data) (GitHub).

References

[1] Ke Mao, Licia Capra, Mark Harman, Yue Jia, A survey of the use of crowdsourcing in software engineering, J. Syst. Softw. 126 (2017) 57–84.

[2] Karim R. Lakhani, David A. Garvin, Eric Lonstein, Topcoder (a): Developing software through crowdsourcing, 2010, Harvard Business School General Management Unit Case (610–032).

[3] Thomas D. LaToza, W. Ben Towne, André Van Der Hoek, James D Herbsleb, Crowd development, in: 2013 6th International Workshop on Cooperative and Human Aspects of Software Engineering, CHASE, IEEE, 2013, pp. 85–88.

[4] Klaas-Jan Stol, Brian Fitzgerald, Two's company, three's a crowd: a case study of crowdsourcing software development, in: Proceedings of the 36th International Conference on Software Engineering, 2014, pp. 187–198.

[5] Topcoder, Topcoder statistics, 2024, URL https://www.topcoder.com/community/statistics/?tracks[All-pills]=0&tracks[General]=0. (Online; Accessed 28 April 2024).

[6] uTest, About uTest, 2024, URL https://www.utest.com/about-us. (Online; Accessed 28 April 2024).

[7] Stack Exchange, Stack exchange data explorer, 2024, URL https://data.stackexchange.com/. (Online; Accessed 28 April 2024).

[8] Thomas D. LaToza, Andre Van Der Hoek, Crowdsourcing in software engineering: Models, motivations, and challenges, IEEE Softw. 33 (1) (2015) 74–80.

[9] Casey Casalnuovo, Bogdan Vasilescu, Premkumar Devanbu, Vladimir Filkov, Developer onboarding in GitHub: the role of prior social links and language experience, in: Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering, 2015, pp. 817–828.

[10] Tadej Matek, Svit Timej Zebec, GitHub open source project recommendation system, 2016, arXiv preprint arXiv:1602.02594.

[11] Cheng Yang, Qiang Fan, Tao Wang, Gang Yin, Huaimin Wang, Repolike: personal repositories recommendation in social coding communities, in: Proceedings of the 8th Asia-Pacific Symposium on Internetware, 2016, pp. 54–62.

[12] Ying Zhen, Abdullah Khan, Shah Nazir, Zhao Huiqi, Abdullah Alharbi, Sulaiman Khan, Crowdsourcing usage, task assignment methods, and crowdsourcing platforms: A systematic literature review, J. Softw.: Evol. Process. 33 (8) (2021) e2368.

[13] Barbara Kitchenham, Stuart Charters, et al., Guidelines for performing systematic literature reviews in software engineering, 2007.

[14] Mark Petticrew, Helen Roberts, Systematic reviews in the social sciences: A practical guide, John Wiley & Sons, 2008.

[15] Eliezer Dutra, Bruna Diirr, Gleison Santos, Human factors and their influence on software development teams-a tertiary study, in: Proceedings of the XXXV Brazilian Symposium on Software Engineering, 2021, pp. 442–451.

[16] Per Lenberg, Robert Feldt, Lars Göran Wallgren, Behavioral software engineering: A definition and systematic literature review, J. Syst. Softw. 107 (2015) 15–37.

[17] David E. Avison, Francis Lau, Michael D. Myers, Peter Axel Nielsen, Action research, Commun. ACM 42 (1) (1999) 94–97.

[18] Laleh Pirzadeh, Human factors in software development: a systematic literature review, 2010.

[19] Lianipng Chen, Muhammad Ali Babar, He Zhang, Towards an evidence-based understanding of electronic data sources, in: 14th International Conference on Evaluation and Assessment in Software Engineering, EASE, 2010, pp. 1–4.

[20] Hira Naveed, Chetan Arora, Hourieh Khalajzadeh, John Grundy, Omar Haggag, Model driven engineering for machine learning components: A systematic literature review, Inf. Softw. Technol. (2024) 107423.

[21] Hashini Gunatilake, John Grundy, Rashina Hoda, Ingo Mueller, The impact of human aspects on the interactions between software developers and end-users in software engineering: A systematic literature review, Inf. Softw. Technol. (2024) 107489.

[22] Lori B. Shelby, Jerry J. Vaske, Understanding meta-analysis: A review of the methodological literature, Leis. Sci. 30 (2) (2008) 96–110.

[23] Virginia Braun, Victoria Clarke, Using thematic analysis in psychology, Qual. Res. Psychol. 3 (2) (2006) 77–101.

[24] Muhammad Zahid Tunio, Haiyong Luo, Wang Cong, Zhao Fang, Abdul Rehman Gilal, Ahsanullah Abro, Shao Wenhua, Impact of personality on task selection in crowdsourcing software development: A sorting approach, IEEE Access 5 (2017) 18287–18294.

[25] Abdul Rehman Gilal, Muhammad Zahid Tunio, Ahmad Waqas, Malek Ahmad Almomani, Sajid Khan, Ruqaya Gilal, Task assignment and personality: Crowdsourcing software development, in: Research Anthology on Agile Software, Software Development, and Testing, IGI Global, 2022, pp. 1795–1809.

[26] Deepjyoti Roy, Mala Dutta, A systematic review and research perspective on recommender systems, J. Big Data 9 (1) (2022) 59.

[27] Mona Taghavi, Jamal Bentahar, Kaveh Bakhtiyari, Chihab Hanachi, New insights towards developing recommender systems, Comput. J. 61 (3) (2018) 319–348.

[28] Leslie Pack Kaelbling, Michael L. Littman, Andrew W. Moore, Reinforcement learning: A survey, J. Artificial Intelligence Res. 4 (1996) 237–285.

[29] Isabel Briggs Myers, Mary H. McCaulley, Robert Most, Manual: A guide to the development and use of the myers-briggs type indicator, 1985.

[30] Luiz Fernando Capretz, Daniel Varona, Arif Raza, Influence of personality types in software tasks choices, Comput. Hum. Behav. 52 (2015) 373–378.

[31] Zainab Umair Kamangar, Umair Ayaz Kamangar, Qasim Ali, Isma Farah, Shahzad Nizamani, Tauha Hussain Ali, To enhance effectiveness of crowdsource software testing by applying personality types, in: Proceedings of the 8th International Conference on Software and Information Engineering, 2019, pp. 15–19.

[32] Kumar Abhinav, Gurpriya Kaur Bhatia, Alpana Dubey, Sakshi Jain, Nitish Bhardwaj, TasRec: a framework for task recommendation in crowdsourcing, in: Proceedings of the 15th International Conference on Global Software Engineering, 2020, pp. 86–95.

[33] Fabio Santos, Jacob Penney, Joao Felipe Pimentel, Igor Wiese, Igor Steinmacher, Marco A. Gerosa, Tell me who are you talking to and I will tell you what issues need your skills, 2023.

[34] GitHub, Inc., GitHub, 2024, https://github.com/about. (Accessed: 16 May 2024).

[35] Topcoder, Topcoder, 2024, https://www.topcoder.com/about-us/. (Accessed: 16 May 2024).

[36] The Bugzilla Project, Bugzilla, 2024, https://www.bugzilla.org/about/. (Accessed: 16 May 2024).

[37] Stack Overflow, Stack Overflow, 2024, https://stackoverflow.com/tour. (Accessed: 16 May 2024).

[38] ChinaSoft International, JointForce, 2024, https://www.jfh.com/en/aboutus.html. (Accessed: 16 May 2024).

[39] Amazon, Amazon Mechanical Turk, 2024, https://www.mturk.com/. (Accessed: 16 May 2024).

[40] Testin, Testin, 2024, https://www.testin.net/. (Accessed: 16 May 2024).

[41] Pierre Bourque, Robert Dupuis, Alain Abran, James W. Moore, Leonard Tripp, Karen Shyne, Bryan Pflug, Marcela Maya, Guy Tremblay, Guide to the software engineering body of knowledge-a straw man version, 1998.

[42] Bill Curtis, Herb Krasner, Neil Iscoe, A field study of the software design process for large systems, Commun. ACM 31 (11) (1988) 1268–1287.

[43] Tamunoemi Fancey Ockiya, Russell Lock, A review of human factors in remote software project management: A progressive look at human based issues in remote software development environments, in: Proceedings of the 2023 12th International Conference on Software and Information Engineering, 2023, pp. 15–21.

[44] Shahzad Sarwar Bhatti, Xiaofeng Gao, Guihai Chen, General framework, opportunities and challenges for crowdsourcing techniques: A comprehensive survey, J. Syst. Softw. 167 (2020) 110611.

[45] Mahmoud Hosseini, Keith T Phalp, Jacqui Taylor, Raian Ali, Towards crowdsourcing for requirements engineering, 2014.

[46] Marco Ortu, Bram Adams, Giuseppe Destefanis, Parastou Tourani, Michele Marchesi, Roberto Tonelli, Are bullies more productive? Empirical study of affectiveness vs. issue fixing time, in: 2015 IEEE/ACM 12th Working Conference on Mining Software Repositories, IEEE, 2015, pp. 303–313.

[47] Marco Ortu, Giuseppe Destefanis, Bram Adams, Alessandro Murgia, Michele Marchesi, Roberto Tonelli, The jira repository dataset: Understanding social aspects of software development, in: Proceedings of the 11th International Conference on Predictive Models and Data Analytics in Software Engineering, 2015, pp. 1–4.

[48] Emitza Guzman, Bernd Bruegge, Towards emotional awareness in software development teams, in: Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering, 2013, pp. 671–674.

[49] Xu Yu, Yadong He, Yu Fu, Yu Xin, Junwei Du, Weijian Ni, Cross-domain developer recommendation algorithm based on feature matching, in: Computer Supported Cooperative Work and Social Computing: 14th CCF Conference, ChineseCSCW 2019, Kunming, China, August 16–18, 2019, Revised Selected Papers 14, Springer, 2019, pp. 443–457.

[50] Iván Cantador, Ignacio Fernández-Tobías, Shlomo Berkovsky, Paolo Cremonesi, Cross-domain recommender systems, Recomm. Syst. Handb. (2015) 919–959.

[51] Paolo Cremonesi, Antonio Tripodi, Roberto Turrin, Cross-domain recommender systems, in: 2011 IEEE 11th International Conference on Data Mining Workshops, Ieee, 2011, pp. 496–503.

[52] Shuhan Yan, Beijun Shen, Wenkai Mo, Ning Li, Transfer learning for cross-platform software crowdsourcing recommendation, in: 2017 24th Asia-Pacific Software Engineering Conference, APSEC, IEEE, 2017, pp. 269–278.

[53] Liang Ying, Liu Boqin, Application of transfer learning in task recommendation system, Procedia Eng. 174 (2017) 518–523.

[54] Ming Yan, Jitao Sang, Tao Mei, Changsheng Xu, Friend transfer: Cold-start friend recommendation with cross-platform transfer learning of social knowledge, in: 2013 IEEE International Conference on Multimedia and Expo, ICME, IEEE, 2013, pp. 1–6.

[55] Ming Yan, Jitao Sang, Tao Mei, Changsheng Xu, Friend transfer: Cold-start friend recommendation with cross-platform transfer learning of social knowledge, in: 2013 IEEE International Conference on Multimedia and Expo, ICME, 2013, pp. 1–6, http://dx.doi.org/10.1109/ICME.2013.6607510.

[56] Weike Pan, Evan Xiang, Nathan Liu, Qiang Yang, Transfer learning in collaborative filtering for sparsity reduction, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 24, (no. 1) 2010, pp. 230–235.

[57] Bin Li, Qiang Yang, Xiangyang Xue, Transfer learning for collaborative filtering via a rating-matrix generative model, in: Proceedings of the 26th Annual International Conference on Machine Learning, 2009, pp. 617–624.

[58] Hongwei Zhang, Xiangwei Kong, Yujia Zhang, Selective knowledge transfer for cross-domain collaborative recommendation, IEEE Access 9 (2021) 48039–48051.

[59] Arjan J.P. Jeckmans, Michael Beye, Zekeriya Erkin, Pieter Hartel, Reginald L Lagendijk, Qiang Tang, Privacy in recommender systems, Soc. Media Retr. (2013) 263–281.

[60] Weiqing Min, Bing-Kun Bao, Changsheng Xu, M. Shamim Hossain, Cross-platform multi-modal topic modeling for personalized inter-platform recommendation, IEEE Trans. Multimed. 17 (10) (2015) 1787–1801.

[61] Junchen Fu, Fajie Yuan, Yu Song, Zheng Yuan, Mingyue Cheng, Shenghui Cheng, Jiaqi Zhang, Jie Wang, Yunzhu Pan, Exploring adapter-based transfer learning for recommender systems: Empirical studies and practical insights, in: Proceedings of the 17th ACM International Conference on Web Search and Data Mining, 2024, pp. 208–217.

[62] G.K. Suhas, S.N. Devananda, R. Jagadeesh, Piyush Kumar Pareek, Sunanda Dixit, Recommendation-based interactivity through cross platform using big data, in: Emerging Technologies in Data Mining and Information Security: Proceedings of IEMIS 2020, Volume 3, Springer, 2021, pp. 651–659.

[63] Chia-Ling Chang, Yen-Liang Chen, Jia-Shin Li, A cross-platform recommendation system from facebook to instagram, Electron. Libr. 41 (2/3) (2023) 264–285.

[64] Xu Yu, Yadong He, Yu Fu, Yu Xin, Junwei Du, Weijian Ni, Cross-domain developer recommendation algorithm based on feature matching, in: Yuqing Sun, Tun Lu, Zhengtao Yu, Hongfei Fan, Liping Gao (Eds.), Computer Supported Cooperative Work and Social Computing, Springer Singapore, Singapore, 2019, pp. 443–457.

[65] Anusha Kanungo, Shweta Kamath, Simran Gosai, Ravita Mishra, Cross platform recommendation system, in: ICDSMLA 2019: Proceedings of the 1st International Conference on Data Science, Machine Learning and Applications, Springer, 2020, pp. 663–670.

[66] Da Cao, Xiangnan He, Liqiang Nie, Xiaochi Wei, Xia Hu, Shunxiang Wu, Tat-Seng Chua, Cross-platform app recommendation by jointly modeling ratings and texts, ACM Trans. Inf. Syst. (TOIS) 35 (4) (2017) 1–27.

[67] Zhu Sun, Di Yu, Hui Fang, Jie Yang, Xinghua Qu, Jie Zhang, Cong Geng, Are we evaluating rigorously? benchmarking recommendation for reproducible evaluation and fair comparison, in: Proceedings of the 14th ACM Conference on Recommender Systems, 2020, pp. 23–32.

[68] Eva Zangerle, Christine Bauer, Evaluating recommender systems: survey and framework, ACM Comput. Surv. 55 (8) (2022) 1–38.

[69] Alan Said, Domonkos Tikk, Klara Stumpf, Yue Shi, Martha A. Larson, Paolo Cremonesi, Recommender systems evaluation: A 3D benchmark., in: RUE@RecSys, 2012, pp. 21–23.

[70] Alberto Sillitti, Giancarlo Succi, Jelena Vlasenko, Understanding the impact of pair programming on developers attention: a case study on a large industrial experimentation, in: 2012 34th International Conference on Software Engineering, ICSE, IEEE, 2012, pp. 1094–1101.

[71] Alberto Bacchelli, Luca Ponzanelli, Michele Lanza, Harnessing stack overflow for the ide, in: 2012 Third International Workshop on Recommendation Systems for Software Engineering, RSSE, IEEE, 2012, pp. 26–30.

[72] Luca Ponzanelli, Holistic recommender systems for software engineering, in: Companion Proceedings of the 36th International Conference on Software Engineering, 2014, pp. 686–689.

[73] Matthias Schmidmaier, Zhiwei Han, Thomas Weber, Yuanting Liu, Heinrich Hußmann, Real-time personalization in adaptive IDEs, in: Adjunct Publication of the 27th Conference on User Modeling, Adaptation and Personalization, 2019, pp. 81–86.

[74] Claudio Di Sipio, Davide Di Ruscio, Phuong T. Nguyen, Democratizing the development of recommender systems by means of low-code platforms, in: Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings, 2020, pp. 1–9.

[75] Dan Zhao, Shauvik Roy Choudhary, Alessandro Orso, Developing analysis and testing plug-ins for modern IDEs: an experience report, Softw.: Pr. Exp. 43 (4) (2013) 465–478.

[76] Robert Chatley, Predictable Dynamic Plugin Architectures (Ph.D. thesis), Imperial College, 2005.

[77] Marko Gasparic, Gail C. Murphy, Francesco Ricci, A context model for IDE-based recommendation systems, J. Syst. Softw. 128 (2017) 200–219.

[78] Yong Liu, Yingtai Xiao, Qiong Wu, Chunyan Miao, Juyong Zhang, Binqiang Zhao, Haihong Tang, Diversified interactive recommendation with implicit feedback, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, (no. 04) 2020, pp. 4932–4939.

[79] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, et al., A survey on large language models for recommendation, World Wide Web 27 (5) (2024) 60.

[80] Chen He, Denis Parra, Katrien Verbert, Interactive recommender systems: A survey of the state of the art and future research challenges and opportunities, Expert Syst. Appl. 56 (2016) 9–27.

[81] Hau Xuan Pham, Jason J. Jung, Preference-based user rating correction process for interactive recommendation systems, Multimedia Tools Appl. 65 (2013) 119–132.

[82] Tariq Mahmood, Francesco Ricci, Learning and adaptivity in interactive recommender systems, in: Proceedings of the Ninth International Conference on Electronic Commerce, 2007, pp. 75–84.

[83] Negar Hariri, Bamshad Mobasher, Robin Burke, Context adaptation in interactive recommender systems, in: Proceedings of the 8th ACM Conference on Recommender Systems, 2014, pp. 41–48.

[84] Wenqi Fan, Zihuai Zhao, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Jiliang Tang, Qing Li, Recommender systems in the era of large language models (llms), 2023, arXiv preprint arXiv:2307.02046.

[85] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Xiangyang Li, Chenxu Zhu, Huifeng Guo, Yong Yu, Ruiming Tang, et al., How can recommender systems benefit from large language models: A survey, 2023, arXiv preprint arXiv:2306.05817.

[86] Junjie Zhang, Ruobing Xie, Yupeng Hou, Xin Zhao, Leyu Lin, Ji-Rong Wen, Recommendation as instruction following: A large language model empowered recommendation approach, ACM Trans. Inf. Syst. (2023).

[87] Joseph Vargovich, Fabio Santos, Jacob Penney, Marco A. Gerosa, Igor Steinmacher, Givemelabeledissues: An open source issue recommendation system, 2023, arXiv preprint arXiv:2303.13418.

[88] Junjie Wang, Ye Yang, Song Wang, Chunyang Chen, Dandan Wang, Qing Wang, Context-aware personalized crowdtesting task recommendation, IEEE Trans. Softw. Eng. 48 (8) (2021) 3131–3144.

[89] Ralph Samer, Alexander Felfernig, Martin Stettinger, Towards issue recommendation for open source communities, in: IEEE/WIC/ACM International Conference on Web Intelligence, 2019, pp. 164–171.

[90] Yueshen Xu, Yuhong Jiang, Xinkui Zhao, Ying Li, Rui Li, Personalized repository recommendation service for developers with multi-modal features learning, in: 2023 IEEE International Conference on Web Services, ICWS, IEEE, 2023, pp. 455–464.

[91] Manasa Golla, B. Eswara Reddy, Project recommendation for open source communities, in: 2022 Fourth International Conference on Emerging Research in Electronics, Computer Science and Technology, ICERECT, IEEE, 2022, pp. 1–6.

[92] Wenyuan Xu, Xiaobing Sun, Jiajun Hu, Bin Li, REPERSP: recommending personalized software projects on GitHub, in: 2017 IEEE International Conference on Software Maintenance and Evolution, ICSME, IEEE, 2017, pp. 648–652.

[93] Ning Li, Wenkai Mo, Beijun Shen, Task recommendation with developer social network in software crowdsourcing, in: 2016 23rd Asia-Pacific Software Engineering Conference, APSEC, IEEE, 2016, pp. 9–16.

[94] Christoph Stanik, Lloyd Montgomery, Daniel Martens, Davide Fucci, Walid Maalej, A simple nlp-based approach to support onboarding and retention in open source communities, in: 2018 IEEE International Conference on Software Maintenance and Evolution, ICSME, IEEE, 2018, pp. 172–182.

[95] Jiaqiu Wang, Zhongjie Wang, Jin Li, A recommendation method for social collaboration tasks based on personal social preferences, IEEE Access 6 (2018) 45206–45216.

[96] Xinli Yang, David Lo, Xin Xia, Lingfeng Bao, Jianling Sun, Combining word embedding with information retrieval to recommend similar bug reports, in: 2016 IEEE 27Th International Symposium on Software Reliability Engineering, ISSRE, IEEE, 2016, pp. 127–137.

[97] Jungil Kim, Geunho Choi, Eunjoo Lee, Semantic similarity-based contributable task identification for new participating developers, J. Inf. Commun. Converg. Eng. 16 (4) (2018) 228–234.

[98] Yu Yang, Wenkai Mo, Beijun Shen, Yuting Chen, Cold-start developer recommendation in software crowdsourcing: A topic sampling approach., in: SEKE, 2017, pp. 376–381.

[99] Runtao Qiao, Shuhan Yan, Beijun Shen, A reinforcement learning solution to cold-start problem in software crowdsourcing recommendations, in: 2018 IEEE International Conference on Progress in Informatics and Computing, PIC, IEEE, 2018, pp. 8–14.

[100] HaiMing Lin, Guanyu Liang, Yanjun Wu, Bin Wu, Chunqi Tian, Wei Wang, Open source software supply chain recommendation based on heterogeneous information network, in: International Symposium on Benchmarking, Measuring and Optimization, Springer, 2022, pp. 70–86.

[101] Zhifang Liao, Shuyuan Cao, Bin Li, Shengzong Liu, Yan Zhang, Song Yu, Graph convolutional network-based repository recommendation system, Comput. Model. Eng. Sci. 137 (1) (2023) 175–196.

[102] Sarah Sayce, Krishnendu Ghosh, Recommendation system for open source projects for minimizing abandonment, in: The International FLAIRS Conference Proceedings, vol. 35, 2022.

[103] Huihui Zhu, Pu Peng, Huan Xu, Open-source project recommendation model, in: E3S Web of Conferences, vol. 268, EDP Sciences, 2021, p. 01061.

[104] Abdulkadir Şeker, Banu Diri, Halil Arslan, New developer metrics for open source software development challenges: An empirical study of project recommendation systems, Appl. Sci. 11 (3) (2021) 920.

[105] Bin Su, Kai Zheng, Wei Wang, A GitHub project recommendation model based on self-attention sequence, in: The 2021 3rd International Conference on Big Data Engineering, 2021, pp. 110–116.

[106] Huan Yang, Song Sun, Junhao Wen, Haini Cai, Muhammad Mateen, Improving personalized project recommendation on GitHub based on deep matrix factorization, in: Collaborative Computing: Networking, Applications and Worksharing: 17th EAI International Conference, CollaborateCom 2021, Virtual Event, October 16-18, 2021, Proceedings, Part I 17, Springer, 2021, pp. 318–332.

[107] Xiaobing Sun, Wenyuan Xu, Xin Xia, Xiang Chen, Bin Li, Personalized project recommendation on GitHub, Sci. China Inf. Sci. 61 (2018) 1–14.

[108] Wenyuan Xu, Xiaobing Sun, Xin Xia, Xiang Chen, Scalable relevant project recommendation on GitHub, in: Proceedings of the 9th Asia-Pacific Symposium on Internetware, 2017, pp. 1–10.

[109] Katsunori Fukui, Tomoki Miyazaki, Masao Ohira, Suggesting questions that match each user's expertise in community question and answering services, in: 2019 20th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing, SNPD, IEEE, 2019, pp. 501–506.

[110] Reza Hadi Mogavi, Sujit Gujar, Xiaojuan Ma, Pan Hui, Hrcr: Hidden markov-based reinforcement to reduce churn in question answering forums, in: PRICAI 2019: Trends in Artificial Intelligence: 16th Pacific Rim International Conference on Artificial Intelligence, Cuvu, Yanuca Island, Fiji, August 26–30, 2019, Proceedings, Part I 16, Springer, 2019, pp. 364–376.

[111] Yiyang Fu, Benjun Shen, Yuting Chen, Linpeng Huang, TDMatcher: A topic-based approach to task-developer matching with predictive intelligence for recommendation, Appl. Soft Comput. 110 (2021) 107720.

[112] Kumar Abhinav, Gurpriya Kaur Bhatia, Alpana Dubey, Sakshi Jain, Nitish Bhardwaj, CrowdAssist: A multidimensional decision support system for crowd workers, J. Softw.: Evol. Process. 35 (6) (2023) e2404.

[113] Denae Ford, Nischal Shrestha, Thomas Zimmermann, ReBOC: Recommending bespoke open source software projects to contributors, in: 2022 IEEE Symposium on Visual Languages and Human-Centric Computing, VL/HCC, IEEE, 2022, pp. 1–5.

[114] Xiuting Ge, Shengcheng Yu, Chunrong Fang, Qi Zhu, Zhihong Zhao, Leveraging android automated testing to assist crowdsourced testing, IEEE Trans. Softw. Eng. 49 (4) (2022) 2318–2336.

[115] Jyun-Yu Jiang, Pu-Jen Cheng, Wei Wang, Open source repository recommendation in social coding, in: Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2017, pp. 1173–1176.

[116] Yuekai Huang, Junjie Wang, Song Wang, Zhe Liu, Dandan Wang, Qing Wang, Characterizing and predicting good first issues, in: Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM, 2021, pp. 1–12.

[117] Hao Ren, Mingliang Ma, Xiaowei Zhang, Yulu Cao, Changhai Nie, Effective recommendation of cross-project correlated issues based on issue metrics, in: Proceedings of the 14th Asia-Pacific Symposium on Internetware, 2023, pp. 1–1.

[118] Wenxin Xiao, Hao He, Weiwei Xu, Xin Tan, Jinhao Dong, Minghui Zhou, Recommending good first issues in github oss projects, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 1830–1842.

[119] Jianguo Wang, Anita Sarma, Which bug should I fix: helping new developers onboard a new project, in: Proceedings of the 4th International Workshop on Cooperative and Human Aspects of Software Engineering, 2011, pp. 76–79.

[120] Chaogang Fu, User intimacy model for question recommendation in community question answering, Knowl.-Based Syst. 188 (2020) 104844.

[121] Muhammad Rezaul Karim, Ye Yang, David Messinger, Guenther Ruhe, Learn or earn?-intelligent task recommendation for competitive crowdsourced software development, 2018.

[122] Connie Yuen Man-Ching, Irwin King, Leung Kwong Sak, Probabilistic matrix factorization with active learning for quality assurance in crowdsourcing systems, 2015.

[123] Gregor Leban, Semantic tools for improving software development in open source communities., in: ISWC (Posters & Demos), 2013, pp. 97–100.

[124] Jie Dai, Qingshan Li, Hui Xue, Zhao Luo, Yinglin Wang, Siyuan Zhan, Graph collaborative filtering-based bug triaging, J. Syst. Softw. 200 (2023) 111667.

[125] Hongrun Wu, Yutao Ma, Zhenglong Xiang, Chen Yang, Keqing He, A spatial–temporal graph neural network framework for automated software bug triaging, Knowl.-Based Syst. 241 (2022) 108308.

[126] Hao He, Haonan Su, Wenxin Xiao, Runzhi He, Minghui Zhou, GFI-bot: Automated good first issue recommendation on GitHub, in: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2022, pp. 1751–1755.
[127] Yutaro Kashiwa, RAPTOR: Release-aware and prioritized bug-fixing task assignment optimization, in: 2019 IEEE International Conference on Software Maintenance and Evolution, ICSME, IEEE, 2019, pp. 629–633.
[128] Dunhui Yu, Yi Wang, Zhuang Zhou, Software crowdsourcing task allocation algorithm based on dynamic utility, IEEE Access 7 (2019) 33094–33106.
[129] Lingxiao Zhang, Yanzhen Zou, Bing Xie, Zixiao Zhu, Recommending relevant projects via user behaviour: an exploratory study on github, in: Proceedings of the 1st International Workshop on Crowd-Based Software Development Methods and Technologies, 2014, pp. 25–30.
[130] Asmita Yadav, Mohammed Baljon, Shailendra Mishra, Sandeep Kumar Singh, Sharad Saxena, Sunil Kumar Sharma, Developer load balancing bug triage: Developed load balance, Expert Syst. (2022) e13006.
[131] Chaogang Fu, Tracking user-role evolution via topic modeling in community question answering, Inf. Process. Manage. 56 (6) (2019) 102075.
[132] Yun Zhang, David Lo, Pavneet Singh Kochhar, Xin Xia, Quanlai Li, Jianling Sun, Detecting similar repositories on GitHub, in: 2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering, SANER, IEEE, 2017, pp. 13–23.
[133] Mohamed Guendouz, Abdelmalek Amine, Reda Mohamed Hamou, Recommending relevant open source projects on github using a collaborative-filtering technique, Int. J. Open Source Softw. Processes (IJOSSP) 6 (1) (2015) 1–16.
[134] Miika Koskela, Inka Simola, Kostas Stefanidis, Open source software recommendations using github, in: Digital Libraries for Open Knowledge: 22nd International Conference on Theory and Practice of Digital Libraries, TPDL 2018, Porto, Portugal, September 10–13, 2018, Proceedings 22, Springer, 2018, pp. 279–285.
[135] Chao Liu, Dan Yang, Xiaohong Zhang, Baishakhi Ray, Md Masudur Rahman, Recommending github projects for developer onboarding, IEEE Access 6 (2018) 52082–52094.
[136] Yuqi Zhou, Jiawei Wu, Yanchun Sun, Ghtrec: a personalized service to recommend github trending repositories for developers, in: 2021 IEEE International Conference on Web Services, ICWS, IEEE, 2021, pp. 314–323.
[137] JaeWon Kim, JeongA Wi, YoungBin Kim, Sequential recommendations on github repository, Appl. Sci. 11 (4) (2021) 1585.
[138] Phuong T. Nguyen, Juri Di Rocco, Riccardo Rubei, Davide Di Ruscio, CrossSim: Exploiting mutual relationships to detect similar oss projects, in: 2018 44th Euromicro Conference on Software Engineering and Advanced Applications, SEAA, IEEE, 2018, pp. 388–395.
[139] Wenxin Xiao, Jingyue Li, Hao He, Ruiqiao Qiu, Minghui Zhou, Personalized first issue recommender for newcomers in open source projects, in: 2023 38th IEEE/ACM International Conference on Automated Software Engineering, ASE, IEEE, 2023, pp. 800–812.
[140] Peng Yang, Chao Chang, Yong Tang, Crowdsourced testing task assignment based on knowledge graphs, in: 2022 IEEE 22nd International Conference on Software Quality, Reliability and Security, QRS, IEEE, 2022, pp. 663–671.
[141] Zixuan Fu, Chenghua Wang, Jiajie Xu, Graph contextualized self-attention network for software service sequential recommendation, Future Gener. Comput. Syst. 149 (2023) 509–517.
[142] Soohan Abbasi, Zulfiqar Ali, Rajesh Kumar, Madiha Shaikh, Komal Maheshwari, Paras Lal, Sagar Kumar, Relevant projectrecommendation system using developer behavior and project features, Int. J. 10 (5) (2021).
[143] Sheetal Phatangare, Aakash Matkar, Akshay Jadhav, Anish Bonde, et al., CodeCompass: NLP-driven navigation to optimal repositories, in: 2024 4th International Conference on Pervasive Computing and Social Networking, ICPCSN, IEEE, 2024, pp. 393–401.
[144] Xin Shen, Xiangjuan Yao, Dunwei Gong, Huijie Tu, Multi-objective optimization and integrated indicator-driven two-stage project recommendation in time-dependent software ecosystem, Inf. Softw. Technol. 170 (2024) 107433.